COMP2870 Theoretical Foundations: Calculus II

Thomas Ranner

6 March 2026

Table of contents

1 Introduction

1.1 Contents of this submodule

This part of the module will deal with numerical algorithms that involve derivatives and integrals. We will approach these problems using a combination of theoretical ideas and practical solutions, thinking through the lens of real-world applications. As a consequence, to succeed in linear algebra, you will do some programming (using Python) and some pen-and-paper theoretical work, too.

1.1.1 Topics

We will have 6 double lectures, 2 tutorials, and 3 labs. We break up the topics as follows:

Lectures

1. Introduction to ordinary differential equations and Euler's method (week 9, L1)
2. The midpoint method and systems of differential equations (week 9, L2)
3. Introduction to optimisation and partial derivatives (week 10, L1)
4. Nonlinear solvers (week 10, L2)
5. Gradient flow (week 11, L1)
6. Using gradient flow to train a neural network (week 11, L2)

Labs

1. Ordinary differential equations (Week 9)
2. Nonlinear solvers (Week 10)
3. Using gradient flow to train a neural network (Week 11)

Tutorials

1. Ordinary differential equations (Week 11)
2. Optimisation (Week 12)

1.1.2 Learning outcomes

Candidates should be able to:

TODO

1.2 Textbooks and other resources

There are many textbooks and other external resources which could help your learning:

TODO

1.3 Programming

This section of the notes links theoretical material with practical applications. We will have weekly lab sessions where you will see how the methods mentioned in the lecture notes work when implemented. More instructions on how we will do this will be covered in the lab sessions themselves.

An important theoretical aspect of translating the algorithms to computer implementations is the use of floating-point numbers. You will have already met floating-point numbers in your first year studies where you met how computer store numbers. You will already know that computer cannot store every possible real number exactly. The inexactness of floating-point numbers has real consequences when performing computations.

Exploring and understanding the material in () is really helpful for understanding many of the choices that we make when forming methods in linear algebra.

We will make extensive use of `Python` and `numpy` during this section of the module. It may help you to revise some of your notes from last year on these topics.

1.4 The big problems for this part of the module

1.4.1 Differential equations

1.4.2 Nonlinear solvers

1.4.3 Optimisation

1.5 Comments on these notes

There are two versions of these notes available: as an online website and as a pdf. I expect minor adjustments to be made to both throughout the progress of delivering the materials.

You can always find the latest versions at:

TODO

Please either email me (T.Ranner@leeds.ac.uk) or use the github repository to report any corrections.

`This version of the notes is 468a759+dirty.`

2 Introduction to dynamic problems

There are many models and problems where time plays an important role.

Examples TODO

Often these models use the language of derivatives and calculus to express the underlying processes. We will spend the first two lectures of this part of the module covering how we can solve this type of dynamic problem using numerical methods.

As a computer scientist, this will help you in your future studies by introducing:

- important methods for solving interesting problems often found in science and engineering, robotics and games design;
- the idea of splitting up a continuous problem into a finite number of steps which makes a tractable problem for a computer to solve;
- measuring errors and efficiency of approaches with a view to balancing between them.

2.1 Rates of change

We will start with a recap of material that you learnt last year to define the objects we want to discuss. We will do this in a way that's driven by our applications and methods which will follow.

Suppose we know that a person is walking at a *constant* speed of 3 meters per second (m/s). What does that really mean? How far will they travel in:

- 0.1 seconds?
- 0.01 seconds?
- 0.001 seconds?
- 10^{-6} seconds?

What does this tell us about speed?

We have that

$$\text{Distance travelled} = \text{speed} \times \text{time},$$

or equivalently:

$$\text{speed} = \frac{\text{distance travelled}}{\text{time}}.$$

What if the object's speed was not constant? For example, we could define the speed

$$S(t) = -(t-1)(t+0.5). \tag{2.1}$$

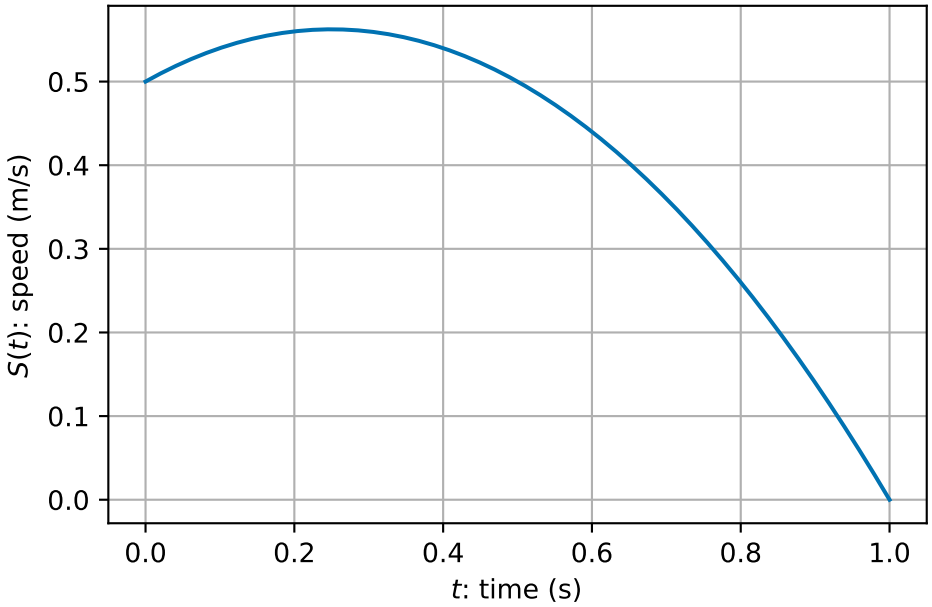


Figure 2.1: A sample object speed over time

How far would the object travel in one second now?

We could consider each tenth of a second separately and estimate the distance covered at each tenth of a second (assuming the $S(t)$ is approximately constant in each interval):

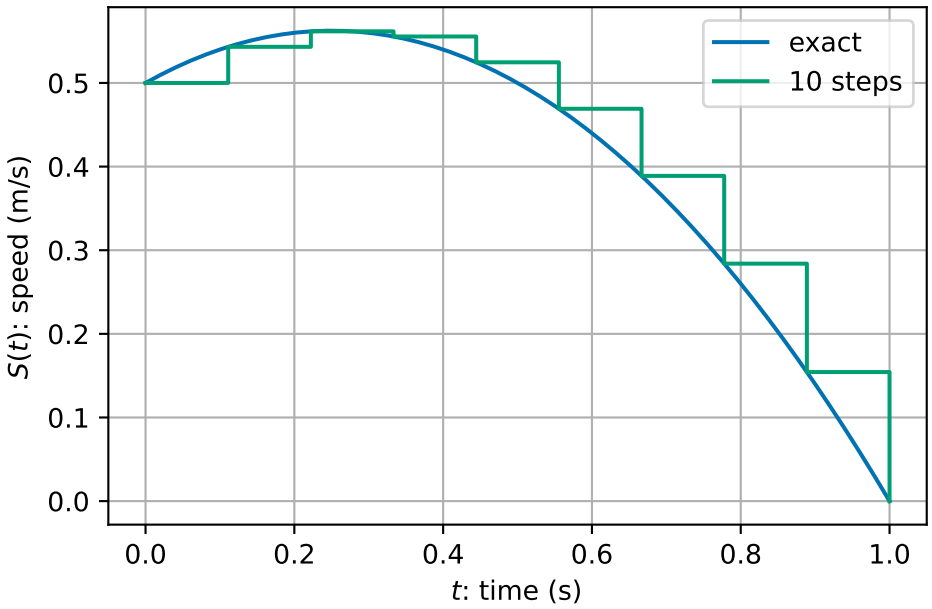


Figure 2.2: A sample object speed over time approximated over 10 steps

```
D, t = 0.0, 0.0

for _i in range(10):
    D = D + 0.1 * S(t)
    t = t + 0.1

print(D, t)
```

```
0.44000000000000006 0.9999999999999999
```

We could consider the same calculation of hundredths of seconds assuming that the speed is approximately constant in each interval:

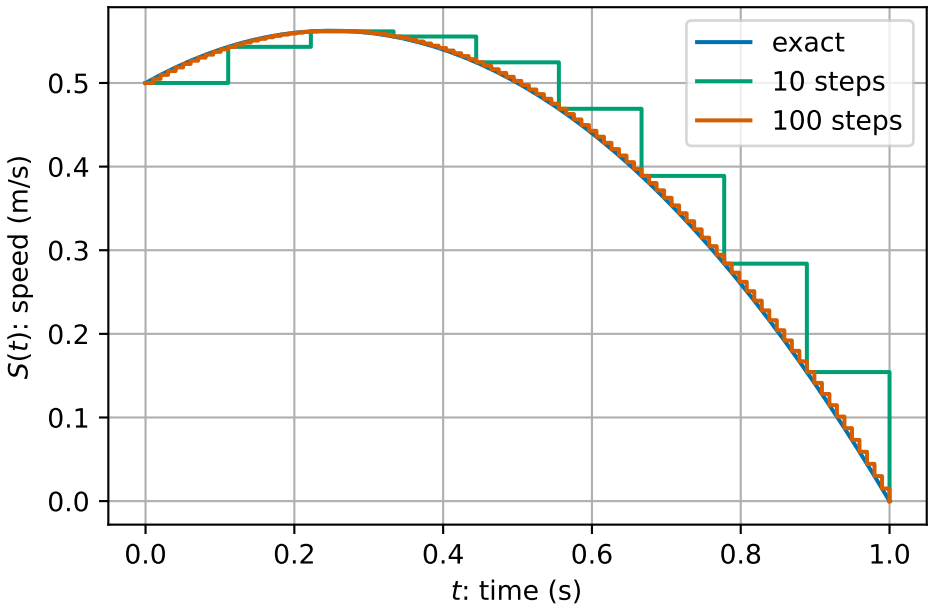


Figure 2.3: A sample object speed over time approximated over 100 steps

```
D, t = 0.0, 0.0

for _i in range(100):
    D = D + 0.01 * S(t)
    t = t + 0.01

print(D, t)
```

```
0.41914999999999963 1.0000000000000007
```

What about if we did a thousand steps

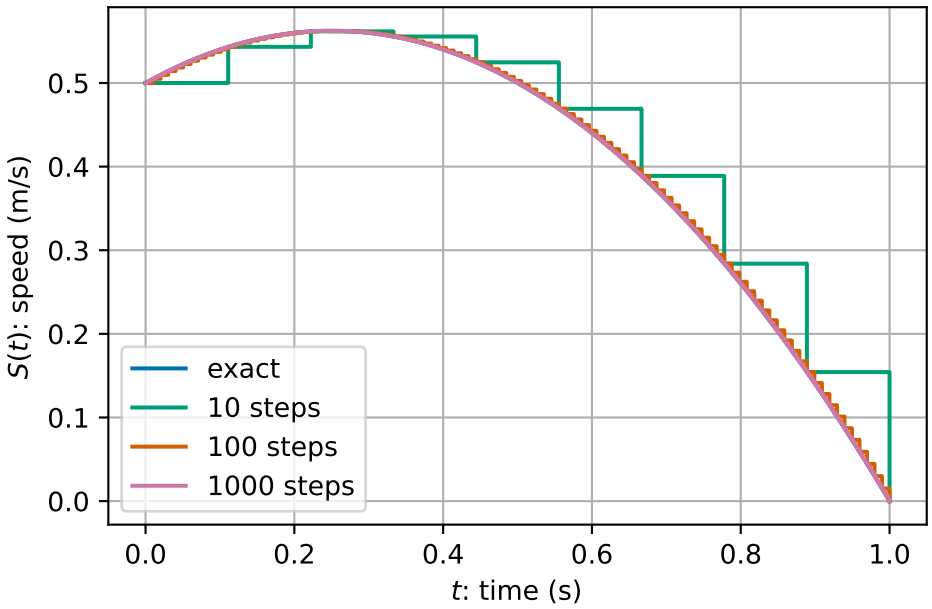


Figure 2.4: A sample object speed over time approximated over 1000 steps

```
D, t = 0.0, 0.0

for _i in range(1000):
    D = D + 0.001 * S(t)
    t = t + 0.001

print(D, t)
```

```
0.41691649999999947 1.0000000000000007
```

In summary, we get

	intervals	increment size, h	total distance
0	1	1.00000	0.500000
1	10	0.10000	0.440000
2	100	0.01000	0.419150
3	1000	0.00100	0.416916
4	10000	0.00010	0.416692
5	100000	0.00001	0.416669

We appear to be converging to a limit as $h \to 0$...

We might express this mathematically as:

$$S(t) = \lim_{h \to 0} \frac{D(t+h) - D(t)}{h},$$

or that *instant* speed is the limit of average speed over smaller and smaller intervals.

Formally, we say that speed, $S(t)$, is the **derivative** of distance, $D(t)$, with respect to time and write $S(t) = D'(t)$.

2.2 Geometric picture

We can plot the expression for the speed, $S(t)$, from (2.1) and it's integral from 0 up to time t, which we call the distance. $D(t)$.

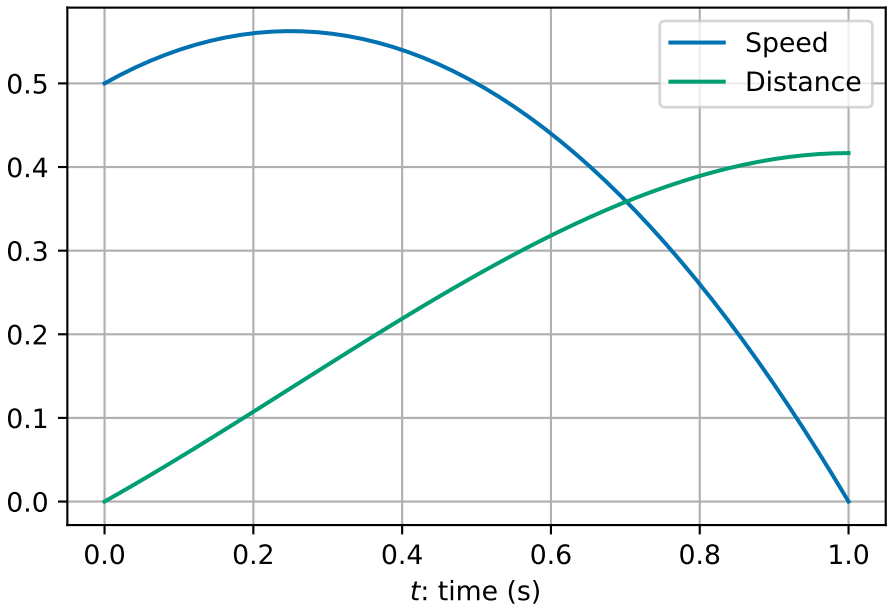


Figure 2.5: Distance and speed examples as a function of time

We see that the **gradient** (slope) of the green curve is the **value** of the blue curve.

- As the blue curve increases in value, the green curve increases in steepness.
- When the blue curve starts to decrease in value, the green curve decreases in steepness.
- By the time the blue curve has a value of zero the green curve is flat.

This gives us an alternative definition of derivative:

> The function $D'(t)$ is the function whose value is equal to the slope (or gradient) of $D(t)$ at every value of t.

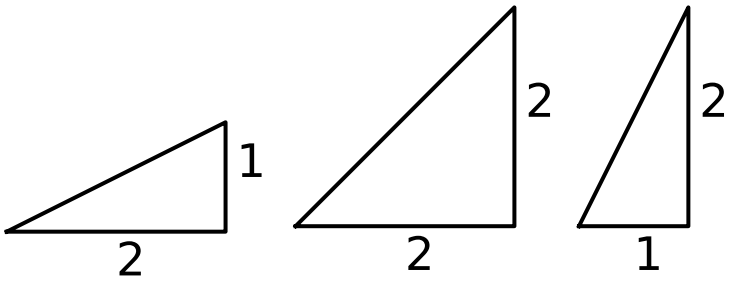


Figure 2.6: Examples of different sloped straight lines.

For a straight line, we can use the formula for a straight line to find the gradient. The equation of a straight line with slope m is given by

$$f(t) = mt + c.$$

For a curve (i.e. non-straight line), we can approximate the slope with a straight line approximation ("chord"):

$$\frac{f(t+h) - f(t)}{h}.$$

We can get a better approximation by taking a smaller value of h…. This leads us back to the definition of derivative (Definition 2.1).

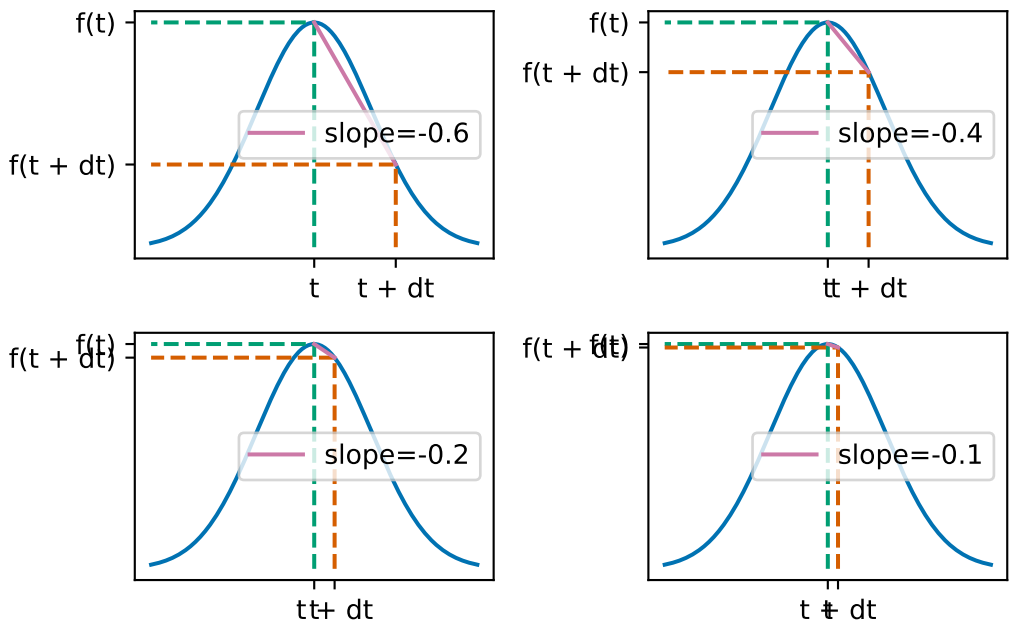


Figure 2.7: Sample straight-line approximations to the slope of a curve.

We can apply similar understanding no matter what the underlying function is. For example, we could do the same things with more complicated real world data. Here is an example using climate data (data source: Met Office).

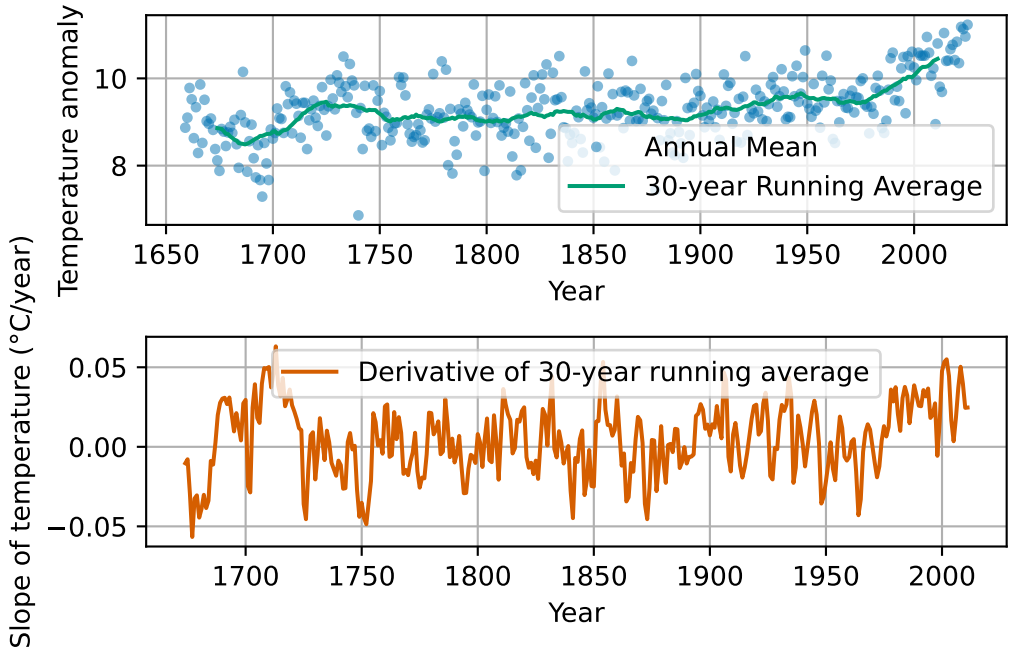


Figure 2.8: Central England Temperature anomaly. Annual Average 1659-2025.

2.3 Formulation in terms of formulae

Let's recall some of the more formal ideas that you learnt last year.

Definition 2.1. Let $f\colon (a,b) \to \mathbb{R}$ be a function on an interval (a,b) of the real line. We say that f is **differentiable** if for all $x \in (a,b)$ the limit:

$$\lim_{h\to 0} \frac{f(x+h)-f(x)}{h},$$

exists and is finite. If f is differentiable then we call the function $g\colon (a,b) \to \mathbb{R}$ the **derivative** of f defined by

$$g(x) = \lim_{h\to 0} \frac{f(x+h)-f(x)}{h}.$$

We will write $f'(x) := g(x)$ or $\frac{\mathrm{d}}{\mathrm{d}x} f(x) = g(x)$.

Last year, you learnt some formula for derivatives. First that you can manipulate derivatives

in nice ways:

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}x}(af_1(x)+bf_2(x)) &= a\frac{\mathrm{d}}{\mathrm{d}x}f_1(x)+b\frac{\mathrm{d}}{\mathrm{d}x}f_2(x) && \text{(sum rule)}\\
\frac{\mathrm{d}}{\mathrm{d}x}(f_1(x)f_2(x)) &= \left(\frac{\mathrm{d}}{\mathrm{d}x}f_1(x)\right)f_2(x)+f_1(x)\left(\frac{\mathrm{d}}{\mathrm{d}x}f_2(x)\right) && \text{(product rule)}\\
\frac{\mathrm{d}}{\mathrm{d}x}(f_1(f_2(x))) &= \frac{\mathrm{d}}{\mathrm{d}y}f_1(y)\Big|_{y=f_2(x)}\frac{\mathrm{d}}{\mathrm{d}x}f_2(x) && \text{(chain rule).}
\end{aligned}$$

You also learnt some derivatives of special functions:

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}x}(x^p) &= px^{p-1}\\
\frac{\mathrm{d}}{\mathrm{d}x}\sin(x) &= \cos(x)\\
\frac{\mathrm{d}}{\mathrm{d}x}\cos(x) &= -\sin(x)\\
\frac{\mathrm{d}}{\mathrm{d}x}\exp(x) &= \exp(x).
\end{aligned}$$

Exercise 2.1. Compute the derivatives of the following functions:

1. $f_1(x) = 3x^2 - 5x + 1$
2. $f_2(x) = \sin(x^2)$
3. $f_3(x) = \exp(-x)\sin(2x)$
4. $f_4(x) = \frac{1}{\eta(x)}$ (for an arbitrary function η)
5. $f_5(x) = \tan(x)$. (*Hint:* use the fact that $\tan(x) = \sin(x)/\cos(x)$ then use the your answer for f_4 and the product and chain rules).

You also learnt about the opposite process to taking derivatives - integration - using Riemann sums:

Definition 2.2. Let $f\colon (a,b) \to \mathbb{R}$ be a function on the interval (a,b) of the real line.

Let $x_0 = a < x_1 < ... < x_n = b$ be a partition of the interval (a,b) into n smaller intervals (x_{i-1}, x_i) for $i = 1, \dots, n$. We choose a point c_i in each interval (x_{i-1}, x_i) and let $h = \max_i x_i - x_{i-1}$. We define the **Riemann sum** of f by

$$s_n = \sum_{i=1}^{n} f(c_i)(x_i - x_{i-1}).$$

Then we define the **integral** of f over (a,b) is given by:

$$\int_a^b f(x)\,\mathrm{d}x = \lim_{h\to 0}\sum_{i=1}^{n} f(c_i)(x_i - x_{i-1}).$$

You learnt the following properties of integration:

$$\int_a^b c_1 f_1(x) + c_2 f_2(x)\,\mathrm{d}x = c_1 \int_a^b f_1(x)\,\mathrm{d}x + c_2 \int_a^b f_2(x)\,\mathrm{d}x$$
$$\int_a^c f(x)\,\mathrm{d}x + \int_c^b f(x)\,\mathrm{d}x = \int_a^b f(x)\,\mathrm{d}x.$$

You learnt that using the fundamental theorem of calculus you can compute integrals by *inverting* the derivative process:

$$\int_a^b x^p\,\mathrm{d}x = \left[\frac{1}{p+1}x^{p+1}\right]_{x=a}^{b} = \frac{1}{p+1}(b^{p+1} - a^{p+1}).$$

Exercise 2.2. Find the integrals of the following functions over $(0, 1)$.

1. $f_1(x) = 3x^2 - 5x + 1$.
2. $f_2(x) = x + \frac{1}{x^2}$.

Remark 2.1. The problem of finding distance over the time interval $(0, 1)$, given speed, $S(t)$, given in (2.1), can be solved using integration.

Indeed, using that $D'(t) = S(t)$, the fundamental theorem of calculus tells us that $D(t)$ is the integral of $S(t)$ with respect to time:

$$\begin{aligned}
D(1) &= \int_0^1 S(t)\,\mathrm{d}t = \int_0^1 -(t-1)(t+0.5)\,\mathrm{d}t \\
&= \int_0^1 -t^2 + 0.5t + 0.5\,\mathrm{d}t \\
&= \left[-\frac{1}{3}t^3 + \frac{0.5}{2}t^2 + 0.5t\right]_{t=0}^{1} \\
&= (-\frac{1}{3} \times 1^3 + 0.25 \times 1^2 + 0.5 \times 1) - (-\frac{1}{3} \times 0^3 + 0.25 \times 0^2 + 0.5 \times 0) \\
&= \frac{5}{12} \approx 0.416667.
\end{aligned}$$

The rest of our section work in this week's lectures is to solve similar equations in the case that our formula for $S(t)$ would depend on t and also D.

2.4 Differential equations

We have already seen and solved one differential equation - when we solved for the distance travel as a function of time: $D'(t) = S(t)$. That equation related the derivative of D to a function of time $S(t)$. When a model takes the form of an equation which involves one or more derivatives then it is called a **differential equation**.

Most models of dynamic processes take the form of differential equations including climate models, many models in engineering and science, financial models, physics engines in computer games and many more!

We are interested in equations of the form:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

The data (things given to us) are:

- the initial time t_0
- the initial value of y at the initial time y_0 - we call this the *initial condition*
- the right hand side function f which can be a function of both time and the solution y,

and we must find a function $y\colon [0, T) \to \mathbb{R}$ for some final time T.

We have seen one example above with $f(t, y) = S(t) = -(t-1)(t+0.5)$, $t_0 = 0$ and $y_0 = 0$. We will see more later.

Remark 2.2. One important consideration is that while many clever ways to solve differential equations exactly exist, most differential equations cannot be solved by hand. Thus numerical methods are not only convenient ways to find solutions of differential equations, in many cases they are essential.

The downside of a computational approach arises now since we are *approximating* the true solution. We will cover more on this in the next lecture.

Example 2.1 (Example differential equation - an object in free fall)**.** Consider the following simple model for an object falling from a large height, based on the two assumptions:

1. all objects are attracted downward with an acceleration due to gravity of $9.81\,\mathrm{m/s^2}$;
2. air resistance causes an object to decelerate in proportion to its speed (i.e. the faster it travels the greater the air resistance).

The *net acceleration downwards* (i.e. rate of change of speed, $S'(t)$) is given by the sum of the two forces g and $-kS(t)$:

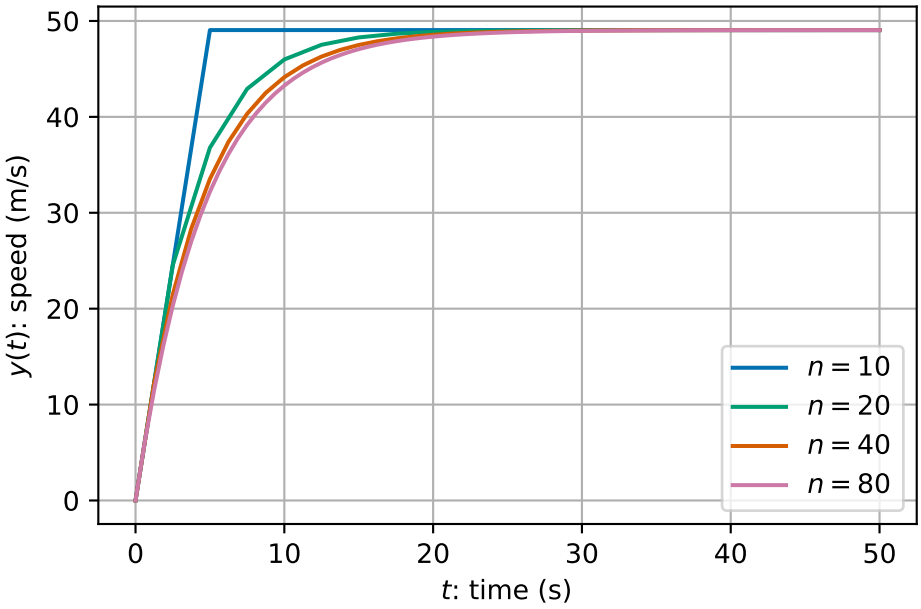
$$S'(t) = g - kS(t).$$

How can we solve this equation?

- We cannot solve this integral exactly:

$$S(t) = S(0) + \int_0^T g - kS(t)\,\mathrm{d}t.$$

- There are techniques to solve this problem analytically (using so-called separation of variables) – but we are simple computer scientists looking for simple computer-based solutions instead!
- We can do similar to the above idea where we divide the time period into lots of small intervals and assume that everything is approximately constant on each time interval…



We see that as we take more time steps, we converge to a smooth solution.

2.5 Euler's method

The approach used in Example 2.1 can be applied to any differential equation in the generic format:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

The general algorithm is very simple:

1. Set initial values $t^{(0)}$ and $y^{(0)}$ from the initial condition.

2. Loop over all time steps, until the final time T, updating using the formula:

$$y^{(i+1)} = y^{(i)} + hf(t^{(i)}, y^{(i)})$$
$$t^{(i+1)} = t^{(i)} + h.$$

The algorithm has one free parameter: the number of time steps, n. Choosing more time steps increases the cost of the algorithm helps us converge to the solution (more on this later). Once the number of time steps is fixed, we can calculate the time step $h = (T - t_0)/(n - 1)$.

Example 2.2. Take two steps of Euler's method to approximate the solution of

$$y'(t) = -(y(t))^2 + 3t^2 \quad \text{subject to the initial condition} \quad y(1) = 2,$$

for $1 \leq t \leq 2$.

In this example we have:

- $n = 2$
- $t_0 = 1$
- $y_0 = 2$
- $T = 2$
- $h = (2 - 1)/2 = 0.5$
- $f(t, y) = -y^2 + 3t^2$.

We start with $t^{(0)} = 1$ and $y^{(0)} = 2$.

- Step 1:

$$\begin{aligned} y^{(1)} &= y^{(0)} + hf(t^{(0)}, y^{(0)}) \\ &= 2 + 0.5(-(2)^2 + 3 \times 1^2) = 1.5 \\ t^{(1)} &= t^{(0)} + h \\ &= 1 + 0.5 = 1.5 \end{aligned}$$

- Step 2:

$$\begin{aligned} y^{(2)} &= y^{(1)} + hf(t^{(1)}, y^{(1)}) \\ &= 1.5 + 0.5(-(1.5)^2 + 3 \times 1.5^2) = 3.75 \\ t^{(2)} &= t^{(1)} + h \\ &= 1.5 + 0.5 = 2.0 \end{aligned}$$

Exercise 2.3. Complete the next two time steps from Example 2.2 to find an approximation of the solution at $T = 3$.

We see that Euler's method is a very general method that is very easy to apply. We can very quickly apply lots of very small time steps for (what we hope will be) a good approximation of the solution. This generality and simplicity is very powerful.

In the next lecture, we will measure the error of this approach and see how we can use our understanding of the errors to find an improve formulation.

2.6 Summary

This lecture introduces you to how we study dynamic problems, situations where things change over time, and why numerical methods are such powerful tools for computer scientists. You have explored familiar ideas like speed and distance, seeing how they connect through derivatives and integrals. From there, the lecture builds up your understanding of what a derivative really means (both as a limit and as a slope), how to compute them, and how integration reverses the process.

Once these foundations are in place, you move into the world of differential equations, which are the backbone of models in physics, engineering, climate science, and even games. Because most real-world equations can't be solved exactly, the lecture shows you how to approximate solutions using Euler's method: a simple, step-by-step numerical technique that computers can apply easily. By the end, you've seen how to turn a continuous, time-dependent problem into something you can compute, setting you up for deeper work on accuracy and error analysis in the next session.

3 Analysis of errors and systems of equations

3.1 Exact derivatives and exact solutions

In a small number of cases, it is possible to solve differential equations *exactly*. That is to write down a simple expression for the function which satisfies both the differential equation and its initial condition. This is rare and is the main reason why we use numerical methods. However, the special cases where we can find exact solutions are useful for testing our numerical methods.

Example 3.1. Consider the function $y(t) = t^3$.

We can find the derivative of $y(t)$:

$$\begin{aligned} y'(t) &= \lim_{h\to 0} \frac{y(t+h) - y(t)}{h} \\ &= \lim_{h\to 0} \frac{(t+h)^3 - t^3}{h} \\ &= \lim_{h\to 0} \frac{t^3 + 3t^2h + 3th^2 + h^3 - t^3}{h} \\ &= \lim_{h\to 0} \frac{3t^2h + 3th^2 + h^3}{h} \\ &= \lim_{h\to 0} 3t^2 + 3th + h^2 \\ &= 3t^2. \end{aligned}$$

By working backwards, we can make up our own differential equation that has $y(t)$ as a known solution. When $y(t) = t^3$, we can compute that

$$y'(t) = 3t^2 = \frac{3y(t)}{t}.$$

Hence, we know the solution to the following equation:

$$y'(t) = \frac{3y(t)}{t} \quad \text{subject to} \quad y(1) = 1. \tag{3.1}$$

If we solve this differential equation for t between 1 and 2, say, then we know that the exact answer when $t = 2$ is $y(2) = 8$.

Note here that we start from $t = 1$ to avoid $t = 0$.

We will use this differential equation where we know the exact solution to test Euler's method. Recall that for a differential equation given by:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

Euler's method is given by:

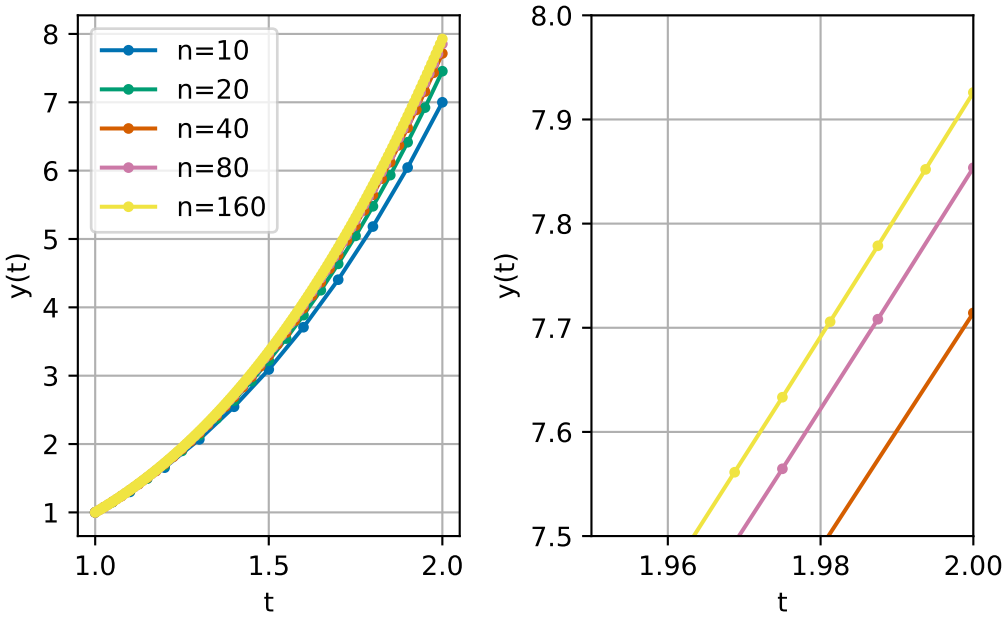
1. Set initial values $t^{(0)}$ and $y^{(0)}$ from the initial condition.
2. Loop over all time steps, until the final time T, updating using the formula:

$$y^{(i+1)} = y^{(i)} + hf(t^{(i)}, y^{(i)})$$
$$t^{(i+1)} = t^{(i)} + h.$$

Exercise 3.1. Compute two time steps of Euler's method to approximate the solution of (3.1) at time $T = 2$. What is the error between your computed solution and the exact solution?

We can also implement Euler's method in code:

```
data = {}

for n in [10, 20, 40, 80, 160]:
    # set up storage for all time steps
    h = (2 - 1) / n
    t, y = np.empty((n + 1,)), np.empty((n + 1,))

    # perform time stepping
    t[0], y[0] = 1.0, 1.0
    for i in range(n):
        t[i + 1], y[i + 1] = t[i] + h, y[i] + h * (3 * y[i]) / t[i]

    # store data
    data[n] = (t, y)
```



We can see that as we increase the number of steps n (reduce the time step h), our solutions become better and better. We can also compute these values using the *absolute error*

$$\text{absolute error} = |y^{(n)} - y(2)|,$$

where $y^{(n)}$ is the solution at the final time step.

Table 3.1: A convergence study for Euler's method

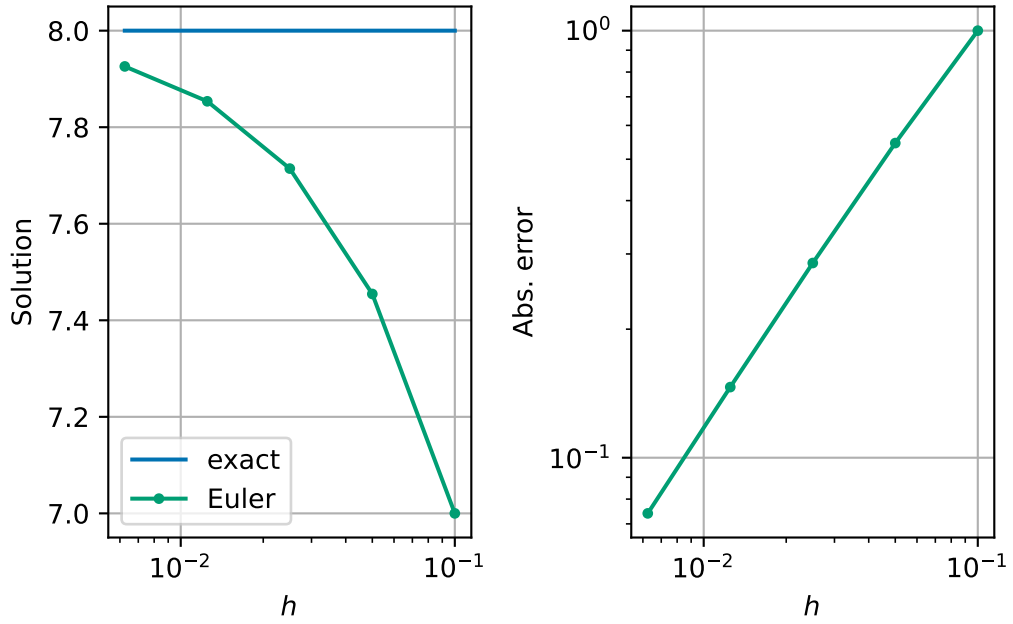
Table 3.1

	n	h	Solution	Abs. error	ratio
0	10	\(1.00000\times 10^{-1}\)	7.000000	\(1.00000\times 10^{0}\)	---
1	20	\(5.00000\times 10^{-2}\)	7.454545	\(5.45455\times 10^{-1}\)	0.545455
2	40	\(2.50000\times 10^{-2}\)	7.714286	\(2.85714\times 10^{-1}\)	0.523810
3	80	\(1.25000\times 10^{-2}\)	7.853659	\(1.46341\times 10^{-1}\)	0.512195
4	160	\(6.25000\times 10^{-3}\)	7.925926	\(7.40741\times 10^{-2}\)	0.506173

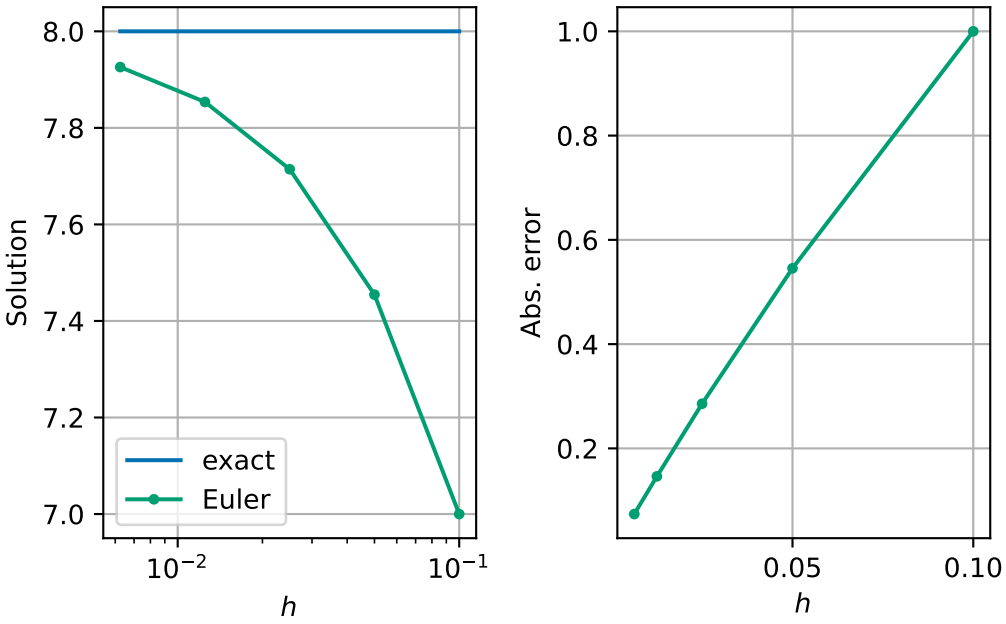
This table shows that indeed the absolute error is decreasing as $h \to 0$. We also compute the ratio between different errors. We see that each time h is halved, that the error is approximately halved. In other words, the error is approximately proportional to h.

Exercise 3.2. What might we expect the computed solution to be if we halved h one more time?

Something interesting happens when we plot h against the absolute error for Euler's method - we see that the errors all fall on a straight line. Perhaps this is not too strange since we think that the error is approximately proportional to step size, so this line should be straight either on a linear or log scale.



Remark 3.1. Using log scales is really important when considering variables like h or error. Contrast the above nice plot, with nice evenly spread data points, with these linear scaled plots:



Exercise 3.3. Why don't we use a log scale for solution too?

Remark 3.2. It is important to note that in this lecture, we are measuring the error at the final time point as a measure of how good our method is. This is a *global error* that measures the accumulation of *local errors* which happen at each time step. Other possible global errors include finding the maximum or average error over all time steps.

Sometimes we also care more about other, more physical based properties of our solution. For example, in long-time simulations scientists often want to ensure that the laws of thermodynamics are implemented exactly – i.e. that energy is conserved and that entropy increases.

3.2 Big O notation

TODO examples from the rest of the module. check this bit with Haiko

When considering algorithm complexity, you have already seen big O notation. For example:

- Gaussian elimination requires $O(n^3)$ operations when n is large,
- Backward substitution requires $O(n^2)$ operations when n is large.

For large values of n the *highest* powers of n are the *most* significant. For smaller values of h, it is the *lowest* powers of h that are the most significant. When $h = 0.001$, h is much bigger than h^2, for example.

We can make use of the big O notation in either case. Formally, we say that $f(h) = O(h^p)$ as $h \to 0$ if there exists a constant $C > 0$ such that $|f(h)| < C|h|^p$ for all small enough h.

Example 3.2. Let $f(x) = 2x^2 + 4x^3 + x^5 + 2x^6$,

- then $f(x) = O(x^6)$ as $x \to \infty$,
- and $f(x) = O(x^2)$ as $x \to 0$.

In this notation, we can say that **the error in Euler's method is $O(h)$.**

3.3 The midpoint method – an improvement on Euler's method

We want to derive a new improved method for solving differential equations. We aim to produce a method with a smaller error for a similar amount of work per time step.

Let's assume that the error in Euler's method is exactly proportional to h. In this case, if we halve h, we halve the error. In this derivation, we will start from the initial condition and want to approximate the solution after a time h. We will label the exact solution at $t = h$ by y_{exact}.

- Then, we continue by computing one time step from $y^{(0)}$ with time step h - we call this solution z_1. We label error between the exact solution and z_1 by E_1.
- We then take two time steps from $y^{(0)}$ now with time step $h/2$ - we call this solution z_2. We label the error between the exact solution and z_2 by E_2. Since in this derivation, the error is exactly proportional to the time step, we can see that $E_2 = \frac{E_1}{2}$.

In equations, we can express these relations as:

$$z_1 - y_{\text{exact}} = E_1$$
$$z_2 - y_{\text{exact}} = E_2 = \frac{E_1}{2}.$$

Subtracting twice the second equation from the first equation gives:

$$y_{\text{exact}} = 2z_2 - z_1.$$

If our assumption that the error in Euler's method is exactly proportional to h, combining z_1 and z_2 would give a solution with zero error! However, our assumption is in fact approximately true as we found in Table 3.1. So we might hope that we have found a method with smaller error than Euler's method with three times the cost of Euler's method (i.e. 3 Euler steps per one time step of this method - one to find z_1 and two to find z_2).

We can even formulate this method more compactly. Writing our new scheme in full, for a general time step i, we have for the one step approach:

$$z_1 = y^{(i)} + hf(t^{(i)}, y^{(i)}),$$

and for the two step approach:

$$\begin{aligned} k &= y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)}) \\ m &= t^{(i)} + \frac{h}{2} \\ z_2 &= k + \frac{h}{2} f(m, k). \end{aligned}$$

Combining z_1 and z_2 as suggested above we see:

$$\begin{aligned} y^{(i+1)} &= 2z_2 - z_1 \\ &= 2\left(k + \frac{h}{2} f(m,k)\right) - \left(y^{(i)} + hf(t^{(i)}, y^{(i)})\right) && \text{(substitute for } z_1 \text{ and } z_2) \\ &= 2k - y^{(i)} + h\left(f(m,k) - f(t^{(i)}, y^{(i)})\right) && \text{(rearrange)} \\ &= 2\left(y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)})\right) - y^{(i)} + h\left(f(m,k) - f(t^{(i)}, y^{(i)})\right) && \text{(substitute for } k) \\ &= 2y^{(i)} - y^{(i)} + h\left(f(t^{(i)}, y^{(i)}) + f(m,k) - f(t^{(i)}, y^{(i)})\right) \\ &= y^{(i)} + hf(m,k). \end{aligned}$$

The midpoint method for solving differential equations is then as follows:

1. Initialise starting values $t^{(0)}$ and $y^{(0)}$.
2. Loop over all time steps, until the final time, updating using the formulae:

$$\begin{aligned} k &= y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)}) \\ m &= t^{(i)} + \frac{h}{2} \\ y^{(i+1)} &= y^{(i)} + hf(m,k) \\ t^{(i+1)} &= t^{(i)} + h. \end{aligned}$$

Remark 3.3. In the end the cost of this formulation of the midpoint method is twice as large as Euler's method.

Example 3.3. Take two steps of the midpoint rule to approximate the solution of

$$y'(t) = -y(t)^2 \quad \text{subject to the initial condition} \quad y(0) = 2,$$

for $0 \leq t \leq 2$.

In this example we have:

- $n = 2$
- $t_0 = 0$
- $y_0 = 2$
- $T = 2$
- $h = (2 - 0)/2 = 1.0$
- $f(t, y) = -y^2$.

We start with $t^{(0)} = 0$ and $y^{(0)} = 2$.

- Step 1:

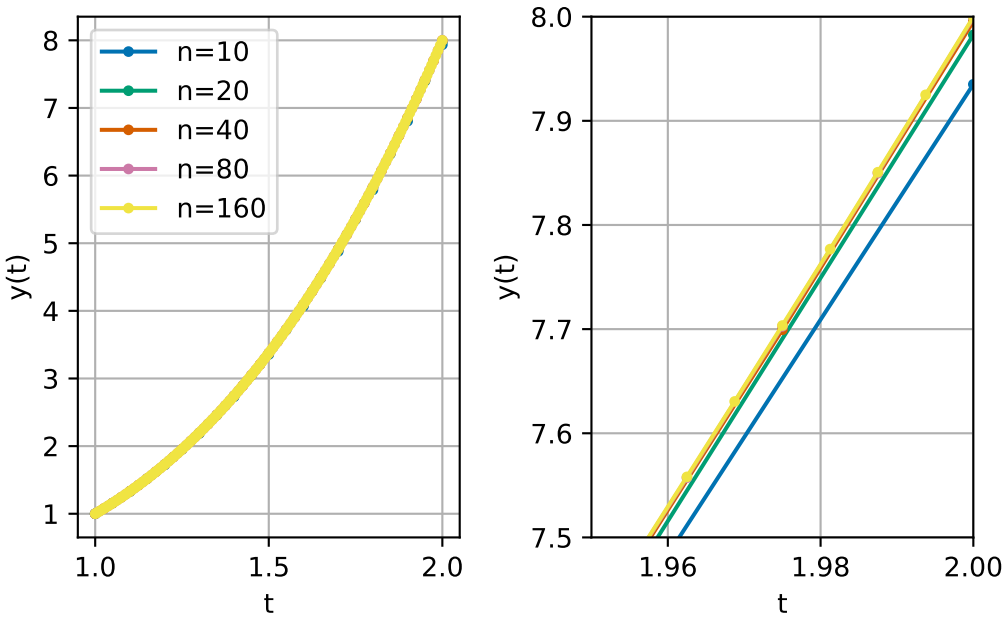
$$\begin{aligned} k &= y^{(0)} + \frac{h}{2} f(t^{(0)}, y^{(0)}) \\ &= 2 + \frac{1.0}{2} \times (-(2)^2) \\ &= 2 + 0.5 \times -4 = 0.0 \\ m &= t^{(0)} + \frac{h}{2} \\ &= 0 + \frac{1.0}{2} = 0.5 \\ y^{(1)} &= y^{(0)} + hf(m, k) \\ &= 2 + 1.0(-(0.0)^2) \\ &= 2 + 1.0 \times -0.0 = 2.0 \\ t^{(1)} &= t^{(0)} + h \\ &= 0 + 1.0 = 1.0. \end{aligned}$$

- Step 2:

$$\begin{aligned} k &= y^{(1)} + \frac{h}{2} f(t^{(1)}, y^{(1)}) \\ &= 2.0 + \frac{1.0}{2} \times (-(2.0)^2) \\ &= 2.0 + 0.5 \times -4.0 = 0.0 \\ m &= t^{(1)} + \frac{h}{2} \\ &= 1.0 + \frac{1.0}{2} = 1.5 \\ y^{(2)} &= y^{(1)} + hf(m, k) \\ &= 2.0 + 1.0(-(0.0)^2) \\ &= 2.0 + 1.0 \times -0.0 = 2.0 \\ t^{(2)} &= t^{(1)} + h \\ &= 1.0 + 1.0 = 2.0. \end{aligned}$$

3.3.1 Numerical results

We can also implement the method in code and explore how the error changes as we change the number of steps. For this test we use the same test as we did for Euler's method (3.1).



We already see that we are much closer to the solution.

Let's compute the errors:

Table 3.2: A convergence study for the midpoint method

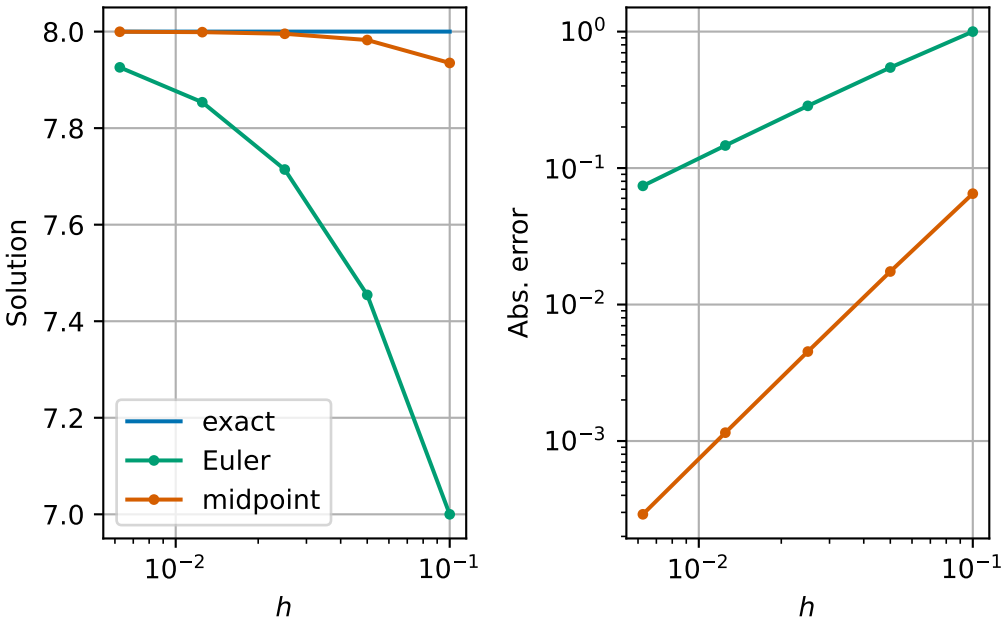
Table 3.2

	n	h	Solution	Abs. error	ratio
0	10	\(1.00000\times 10^{-1}\)	7.935060	\(6.49403\times 10^{-2}\)	---
1	20	\(5.00000\times 10^{-2}\)	7.982538	\(1.74619\times 10^{-2}\)	0.268892
2	40	\(2.50000\times 10^{-2}\)	7.995475	\(4.52485\times 10^{-3}\)	0.259127
3	80	\(1.25000\times 10^{-2}\)	7.998849	\(1.15145\times 10^{-3}\)	0.254473
4	160	\(6.25000\times 10^{-3}\)	7.999710	\(2.90410\times 10^{-4}\)	0.252212

We now see that the error reduces by a *quarter* each time we reduce h by half. This means that the error is approximately proportional to h^2, or equivalently, we can say that the error is $O(h^2)$ as $h \to 0$.

This is a significant improvement on Euler's method. We say that the midpoint method is **second order**, whereas Euler's method is just **first order**.

We can see the significant reduction in error by plotting the errors for both methods:



Our interesting phenomenon of the straight line on the log-log plot of h and absolute error has happened again for the midpoint method. The difference now is that the line is much steeper. In a log-log plot, straight lines correspond to power laws (i.e., Ch^p) and the slope indicates the exponent in the power law (p). Indeed if $\text{error}(h) \approx Ch^p$ then

$$\log(\text{error}(h)) \approx \log C + p \log h.$$

We are seeing that our method behaves as if the error is proportional to h^2.

3.4 Balancing cost and accuracy

At first glance, it may seem that we now have a trade-off between reducing the error but at a higher cost by using the midpoint method. We can test this more explicitly. We repeat the experiments but use two additional measures of cost: the run time and the number of right-hand side (f) evaluations. When working with real-world systems, evaluating the right-hand side function often forms the main computational cost of both Euler's method and the midpoint method.

Table 3.3: A convergence study for Euler's method with running costs

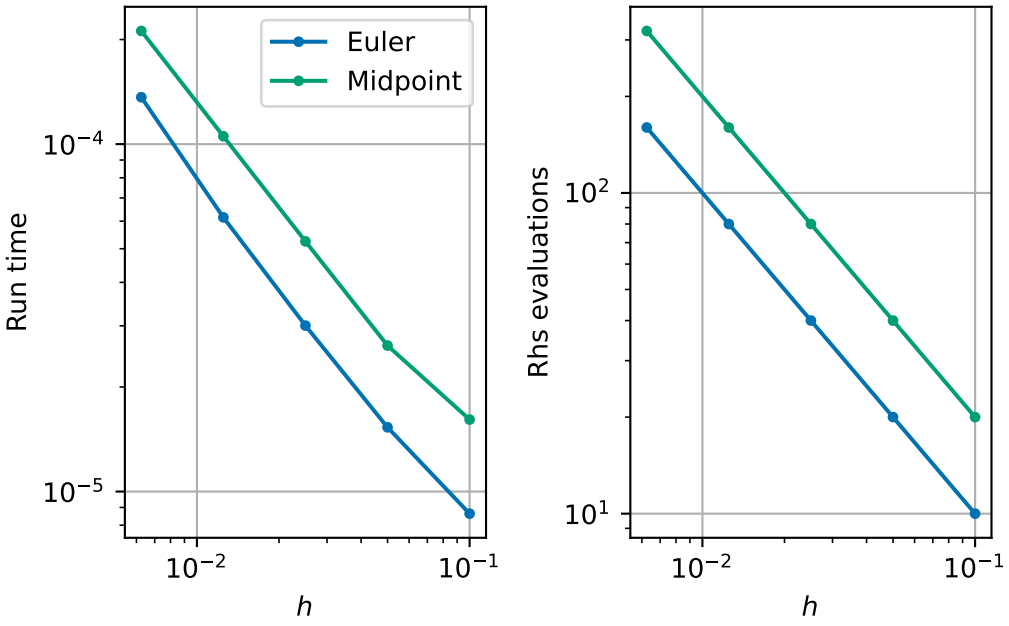
Table 3.3

	n	h	Solution	Abs. error	Run time	Rh
0	10	\(1.00000\times 10^{-1}\)	7.000000	\(1.00000\times 10^{0}\)	\(8.63700\times 10^{-6}\)	10
1	20	\(5.00000\times 10^{-2}\)	7.454545	\(5.45455\times 10^{-1}\)	\(1.53055\times 10^{-5}\)	20
2	40	\(2.50000\times 10^{-2}\)	7.714286	\(2.85714\times 10^{-1}\)	\(3.00350\times 10^{-5}\)	40
3	80	\(1.25000\times 10^{-2}\)	7.853659	\(1.46341\times 10^{-1}\)	\(6.15446\times 10^{-5}\)	80
4	160	\(6.25000\times 10^{-3}\)	7.925926	\(7.40741\times 10^{-2}\)	\(1.36466\times 10^{-4}\)	160

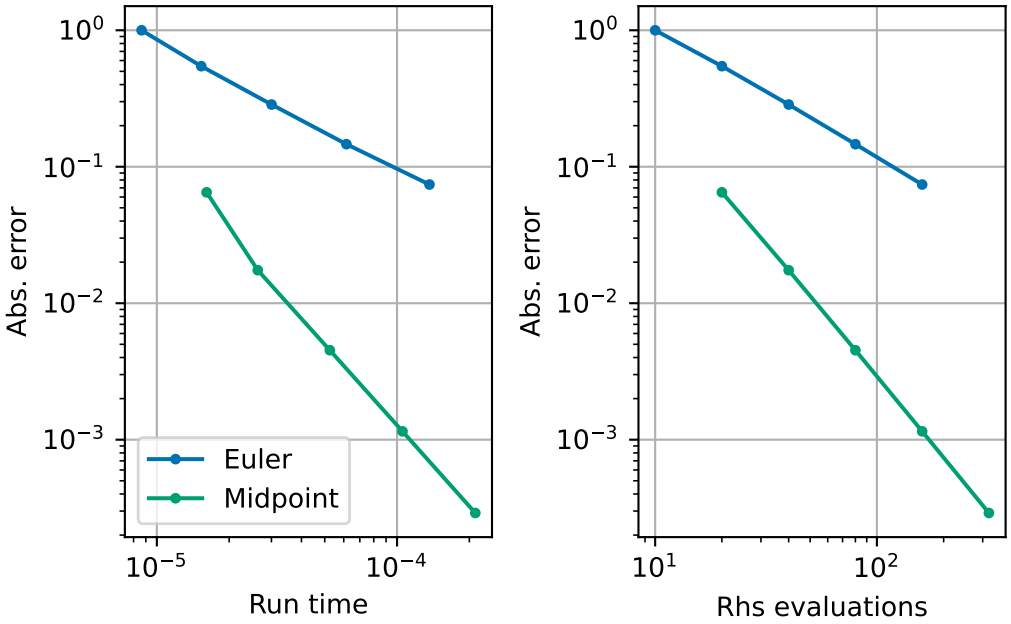
Table 3.4: A convergence study for Midpoint's method with running costs

Table 3.4

	n	h	Solution	Abs. error	Run time	Rh
0	10	\(1.00000\times 10^{-1}\)	7.935060	\(6.49403\times 10^{-2}\)	\(1.61140\times 10^{-5}\)	20
1	20	\(5.00000\times 10^{-2}\)	7.982538	\(1.74619\times 10^{-2}\)	\(2.63150\times 10^{-5}\)	40
2	40	\(2.50000\times 10^{-2}\)	7.995475	\(4.52485\times 10^{-3}\)	\(5.24948\times 10^{-5}\)	80
3	80	\(1.25000\times 10^{-2}\)	7.998849	\(1.15145\times 10^{-3}\)	\(1.05359\times 10^{-4}\)	160
4	160	\(6.25000\times 10^{-3}\)	7.999710	\(2.90410\times 10^{-4}\)	\(2.11742\times 10^{-4}\)	320



However, we can make this plot another way by plotting the cost against the error.



We see now that whichever way you want to measure cost, you see that for a fixed cost the midpoint method gives a lower error. Although the midpoint method costs about twice as much per time step, its higher order accuracy means we need far fewer steps to reach a desired accuracy. So the best way to think is that if you have a budget of, say, right-hand side evaluations, N, you can afford for your computations, then in general, you should choose to use the midpoint method with $n = N/2$ time steps in order to minimise the error.

3.5 Summary

We have found different ways to computationally measure the quality of an algorithm to solve differential equations. We have used one way of measuring errors to derive an improvement to Euler's method called the midpoint method. We see that Euler's method is a first order accurate method and the midpoint method is second order accurate. The computational cost of the midpoint method is twice as high as Euler's method, but the higher order accuracy outweighs the higher computational cost in general.

4 A model for the trajectory of an object

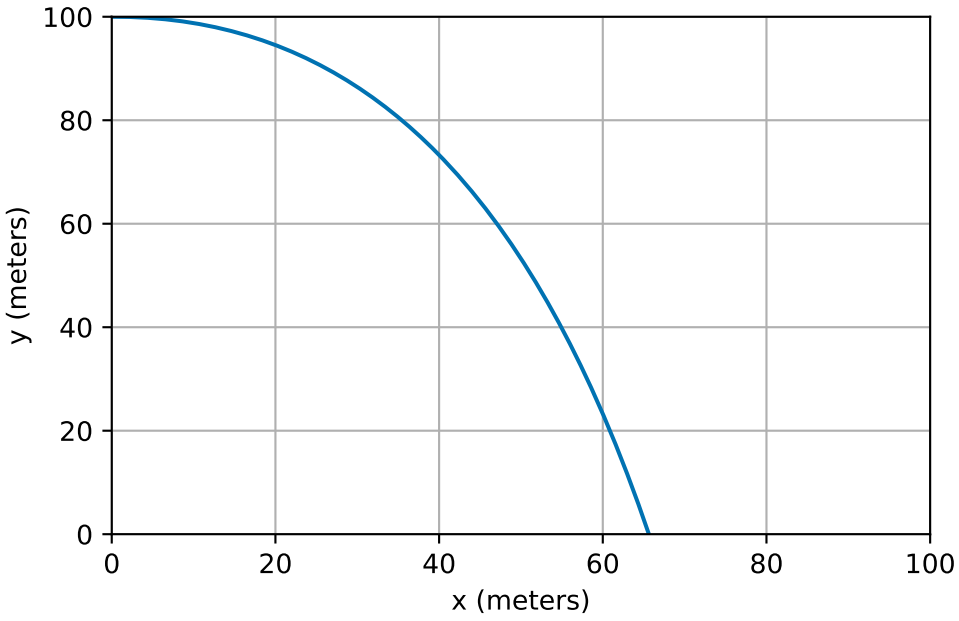
4.1 Newton's laws of motion

A large component of many computer games is the *physics engine*. Planning robotic manoeuvring requires physics-based models too. Both applications typically spend large numbers of CPU cycles simulating the motion of bodies using Newton's laws, based on the forces that are exerted upon them. In particular, Newton's second law of motion says that the net force on an object equals its mass multiplied by its acceleration:

$$\text{Force} = \text{Mass} \times \text{Acceleration}.$$

Once we know the acceleration of an object we can solve differential equations to calculate its subsequent velocity and position…

Consider an object being launched horiztonally from the top of a tall building. We know its initial position and speed in each direction and wish to predict its trajectory.



4.2 A mathematical model

In order to produce a model, we need to decide which are the most important forces acting on the object. One good choice would be to consider gravity and air resistance. Gravity only acts in the negative y direction (not the x direction), and always leads to a constant acceleration ($g = 9.81\,\mathrm{m/s^2}$). Air resistance *opposes* the motion in both x and y directions, and we will assume that the force is proportional to the speed in each direction (similarly to our free fall example, Example 2.1). We will denote the constant of proportionality by k.

We use the following notation:

- $X(t)$ the horizontal distance of the object at time t,
- $Y(t)$ the vertical distance of the object at time t (height above ground, increasing upwards),
- $U(t)$ the horizontal speed of the object at time t,
- $V(t)$ the vertical speed of the object at time t.

The above model leads to the following two differential equations (assuming the mass of the object is 1 kg):

$$\begin{aligned} U'(t) &= -kU(t) \\ V'(t) &= -kV(t) - g. \end{aligned}$$

We also know that, from the definition of speed (rate of change of distance), that

$$\begin{aligned} X'(t) &= U(t) \\ Y'(t) &= V(t). \end{aligned}$$

In addition to these 4 differential equations for the unknown speeds (U and V) and distances (X and Y), we also need to know the initial conditions for the system. We choose one scenario: at $t = 0$,

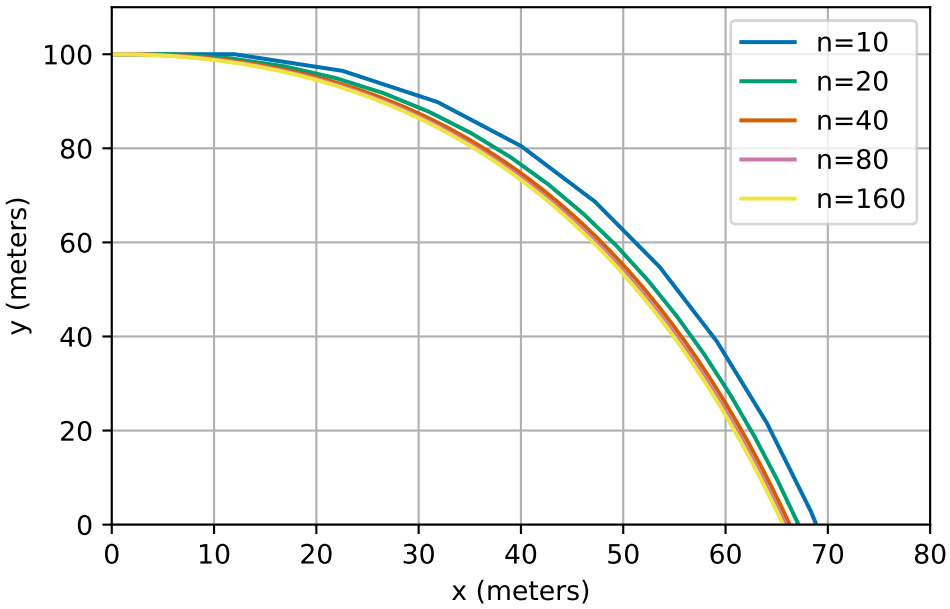
$$\begin{aligned} U(0) &= 20\,\mathrm{m/s} \\ V(0) &= 0\,\mathrm{m/s} \\ X(0) &= 0\,\mathrm{m} \\ Y(0) &= 100\,\mathrm{m}. \end{aligned}$$

From now on we will omit the units in our calculations and assume they are implicit.

4.3 Python implementation

We can implement our model using Euler's method:

```
def projectile(n, T):
    """
    TODO
    """
    # initialise all arrays
    t = np.empty(n + 1)
    U = np.empty(n + 1)
    V = np.empty(n + 1)
    X = np.empty(n + 1)
    Y = np.empty(n + 1)

    # set initial conditions
    t[0] = 0.0
    U[0] = 20.0
    V[0] = 0.0
    X[0] = 0.0
    Y[0] = 100.0

    # define constants
    k = 0.2
    g = 9.81

    # calculate step size
    h = (T - 0) / n

    # perform euler steps
    for i in range(n):
        U[i + 1] = U[i] + h * (-k * U[i])
        V[i + 1] = V[i] + h * (-k * V[i] - g)
        X[i + 1] = X[i] + h * U[i]
        Y[i + 1] = Y[i] + h * V[i]
        t[i + 1] = t[i] + h

    return t, X, Y, U, V
```



4.4 System of equations

The above projectile example shows how to use Euler's method to solve a system of four differential equations. In general, a system of four differential equations will take the form:

$$\begin{aligned} y_1'(t) &= f_1(t, y_1(t), y_2(t), y_3(t), y_4(t)) \\ y_2'(t) &= f_2(t, y_1(t), y_2(t), y_3(t), y_4(t)) \\ y_3'(t) &= f_3(t, y_1(t), y_2(t), y_3(t), y_4(t)) \\ y_4'(t) &= f_4(t, y_1(t), y_2(t), y_3(t), y_4(t)). \end{aligned}$$

Exercise 4.1. How does this general form relate to the projectile example?

It is possible to apply Euler's method or the midpoint method to a general system such as that. These examples may look quite complicated at first, but they are simply applying the same techniques to systems of differential equations rather than a single differential equation.

Example 4.1. Consider the following system of differential equations:

$$\vec{y}\,'(t) = A\vec{y} \quad \text{subject to the initial condition} \quad \vec{y}(0) = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

where $\vec{y}(t) = (y_1(t), y_2(t))^T$ and $A = \begin{pmatrix} -1 & 1 \\ 1 & -2 \end{pmatrix}$. Approximate the solution at time $T = 1$ using 2 steps of Euler's method with $h = 0.5$.

- Step 1:

$$\begin{aligned}
\vec{y}^{(1)} &= \vec{y}^{(0)} + hA\vec{y}^{(0)} \\
&= \begin{pmatrix} 0 \\ 1 \end{pmatrix} + 0.5 \begin{pmatrix} -1 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} \\
&= \begin{pmatrix} 0 \\ 1 \end{pmatrix} + 0.5 \begin{pmatrix} 1 \\ -2 \end{pmatrix} \\
&= \begin{pmatrix} 0.5 \\ 0.0 \end{pmatrix} \\
t^{(1)} &= t^{(0)} + h \\
&= 0 + 0.5 = 0.5.
\end{aligned}$$

- Step 2:

$$\begin{aligned}
\vec{y}^{(2)} &= \vec{y}^{(1)} + hA\vec{y}^{(1)} \\
&= \begin{pmatrix} 0.5 \\ 0.0 \end{pmatrix} + 0.5 \begin{pmatrix} -1 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.0 \end{pmatrix} \\
&= \begin{pmatrix} 0.5 \\ 0.0 \end{pmatrix} + 0.5 \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} \\
&= \begin{pmatrix} 0.25 \\ 0.25 \end{pmatrix} \\
t^{(2)} &= t^{(1)} + h \\
&= 0.5 + 0.5 = 1.0.
\end{aligned}$$

4.5 Summary

We have seen that Euler's method can be applied to systems of differential equation. The same ideas as here can be used to apply the midpoint method to systems of differential equations. Systems of differential equations based on Newton's laws are extremely common in many applications including computer games and robotics.

The methods that we have learnt during this week are a small submit of the many methods available. There are a whole zoo of methods which have specialist properties such as even higher order convergence or preserve physical properties better. You will meet some on the lab sheet this week.

5 Introduction to optimisation

Optimisation is one of the key problems of all numerical computation. It corresponds to many real world problems:

1. When making business decisions, we often want to find a solution which maximises our financial returns;
2. When doing medical procedures, we may want to maximise the effectiveness of the procedure or minimise any potential side effects;
3. When writing computer code, we (normally) want our code to run as fast as possible;
4. When scheduling a train timetable, we may want to maximise the usage of the network, or minimise the cost of running the network, or maximise the robustness to delays of our timetable.
5. When designing a social media app, we want to find the algorithm that encourages people to stay on the app the longest.
6. When controlling a robotic arm, we may want to minimise the time or energy the solution takes.
7. When doing any problem with data, we want to find the model or parameters that best fit the data.
8. When training a neural network, we want to find the parameters of the network to best match the training data.

We make a distinction between problems where the set of possible solutions (admissible states) are continuous or discrete. For example:

1. Choosing between distinct medical procedures is a discrete optimisation problem.
2. Choosing how much of a drug to give is a continuous problem that can be modelled using real numbers.

This part of the module will deal with **continuous optimisation problems**.

All of the continuous optimisation problems above can be expressed in a general mathematical form.

We first need a set $\mathcal{A}$ of admissible states (think of the amount of drug to be delivered or how it is delivered). For continuous problems $\mathcal{A}$ will also be either an interval of the real line $\mathbb{R}$ or a nice subset of the higher dimensional space $\mathbb{R}^n$. Then we can ask what the outcome of that choice is; we model the outcome through a *cost function* (also called potential or loss function), which we label $F\colon \mathcal{A} \to \mathbb{R}$.

Remark. In our examples, where we compute things, we will also have $\mathcal{A} = \mathbb{R}$ or $\mathbb{R}^n$.

The idea of optimisation is to find the best value x^* in $\mathcal{A}$ which minimises F and the value of $F^* = F(x^*)$ of the minimal cost.

Definition 5.1. We will write:

$$\begin{aligned} F^* &= \min_{x \in \mathcal{A}} F(x) \\ x^* &= \arg\min_{x \in \mathcal{A}} F(x). \end{aligned} \tag{5.1}$$

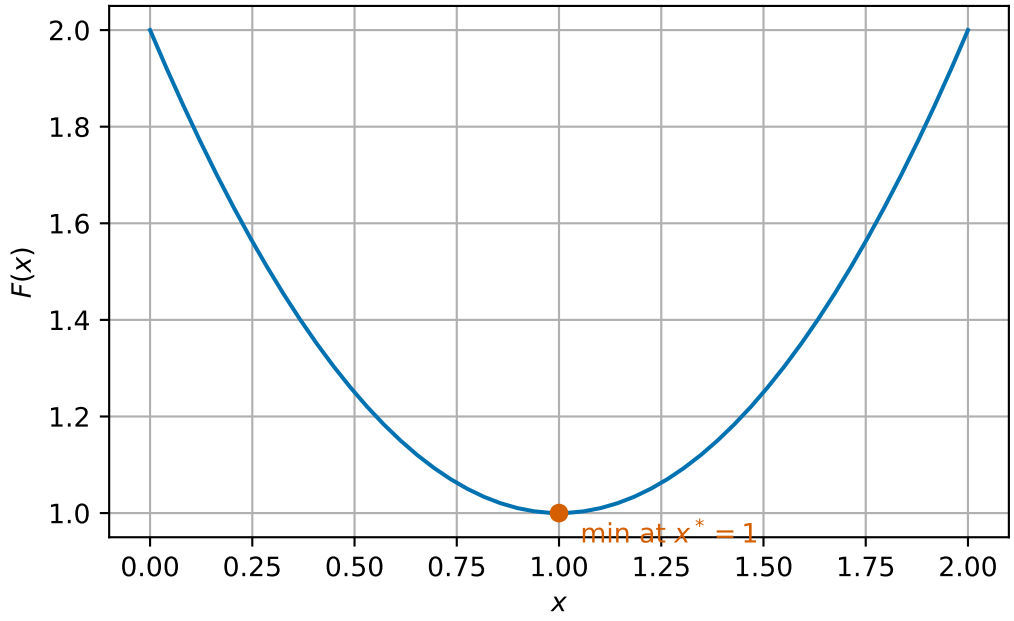
We call x^* the **minimiser** and F^* the **minimising value** or **minimal value**. We note that the argmin may not be a single value but may be a set (containing more than one point). In examples, we often denote one such minimiser by x^*.

5.1 Some interesting examples

It is sometimes easy to see where the minimum of a function is by plotting the function.

In one dimension, we can simply plot the graph of the function. Let $F \colon \mathbb{R} \to \mathbb{R}$ be given by

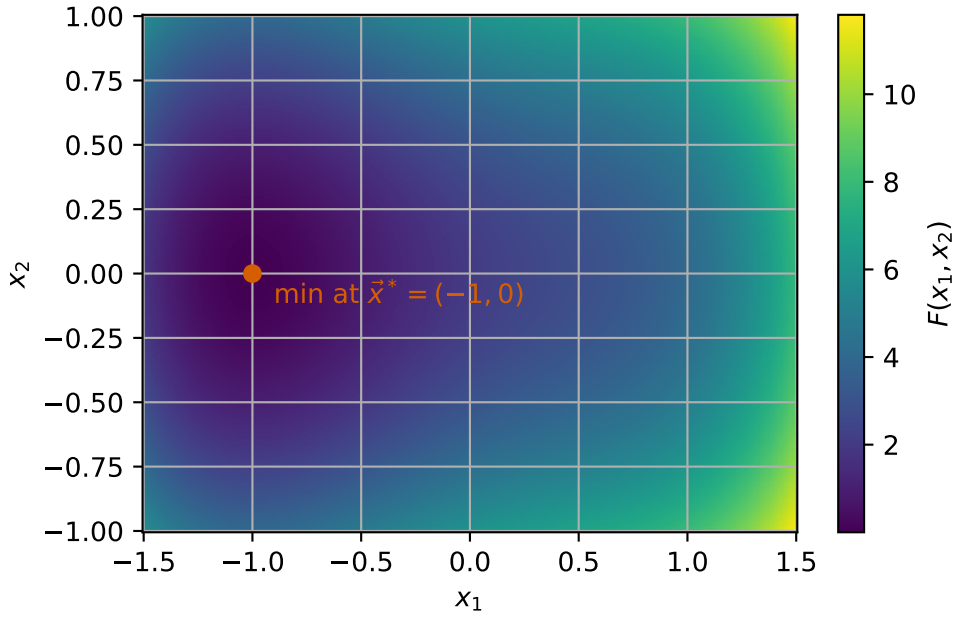
$$F(x) = x^2 - 2x + 2.$$



We can see that the minimum is $F^* = 1$ at $x^* = 1$.

It is hard to visualise functions in higher dimension. One possible idea is to plot slices, where we fix one or more variables at a time to leave a one or two dimensional function. However, this is not often a useful way to help find minimisers. In two dimensions, we can plot a heatmap of a function. This means that for each point in a two dimensional region, we assign a colour based on the value of the function. Each colour corresponds to a different function value. Let $F\colon \mathbb{R}^2 \to \mathbb{R}$ be given by

$$F(\vec{x}) = F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2$$



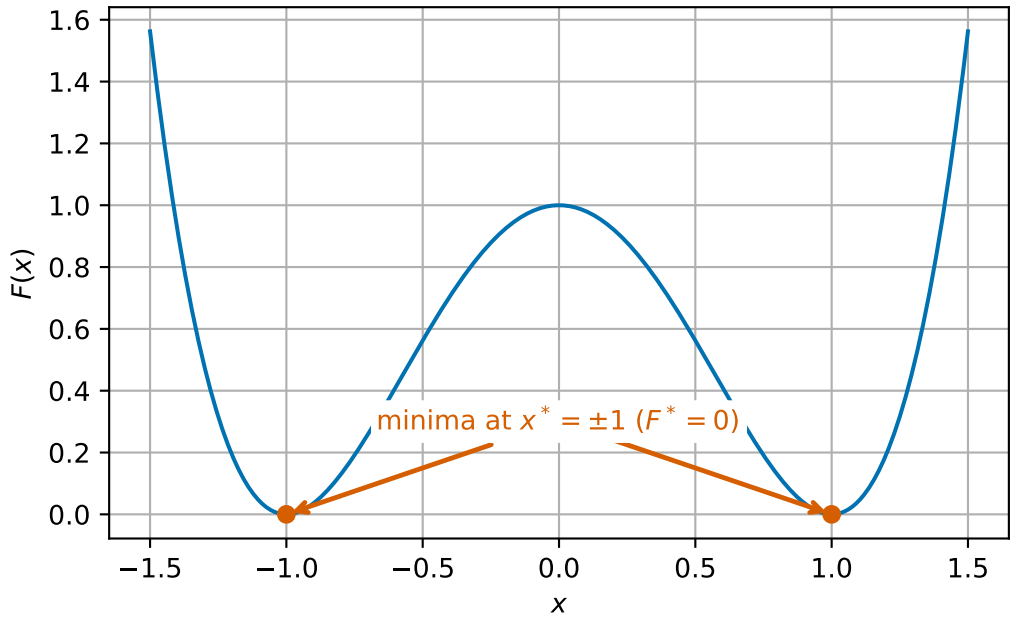
Here we can see a minimum at $\vec{x}^* = (-1, 0)^T$ where $F^* = 0$.

Before we come to some useful ways to find minimisers, we want to explore some other interesting cases.

Sometimes minimisers are not unique. We can have two values which both have the same minimal value. For

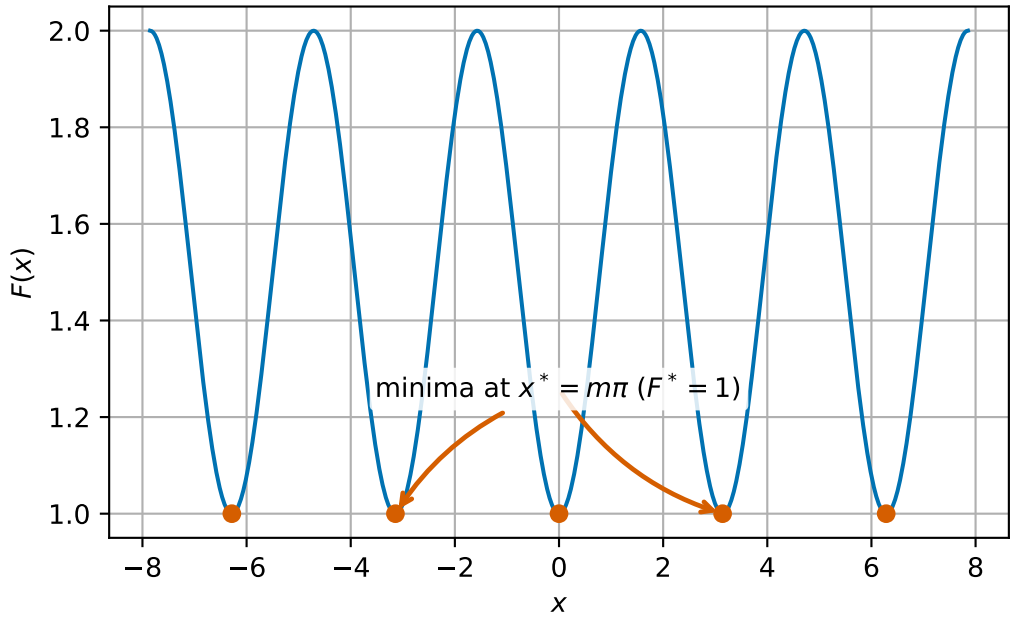
$$F(x) = (x^2 - 1)^2$$

we have two minima at $x^* = -1$ and 1 which both have $F^* = 0$.



Sometimes we can have infinitely many minimisers. Consider

$$F(x) = 1 + \sin^2(x).$$

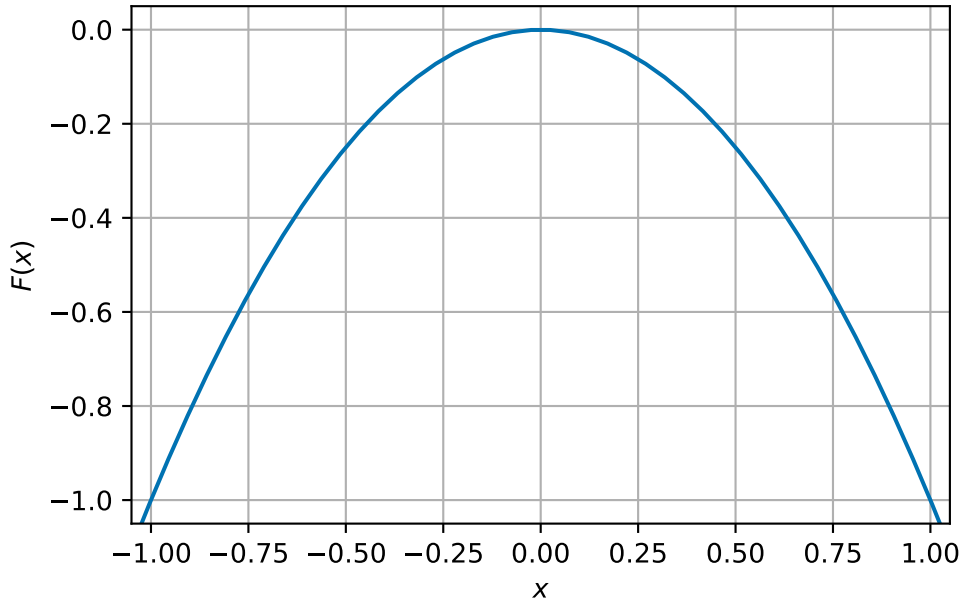


Here, $x^* = m\pi$ for any $m \in \mathbb{Z}$ which all have $F^* = 1$.

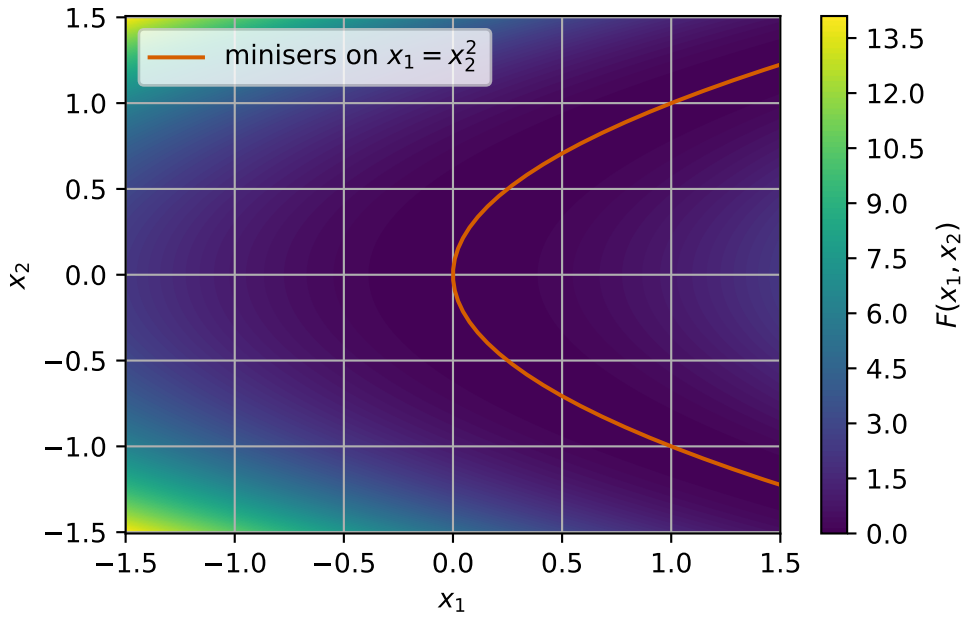
Sometimes, we can have no minimisers at all! For

$$F(x) = -x^2,$$

there is no minimiser in $\mathbb{R}$:



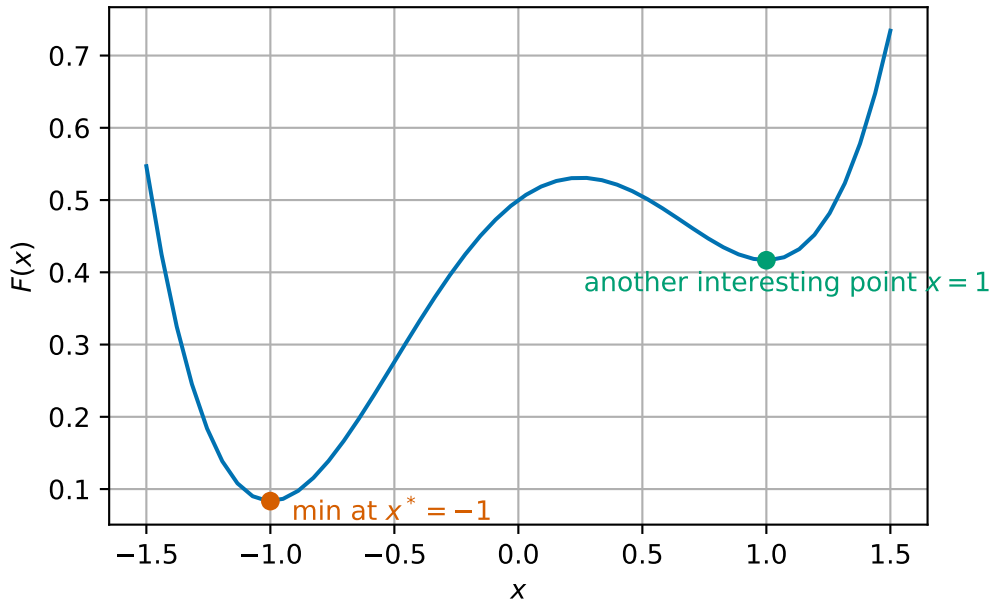
In two (or more) dimensions, the same things can happen. We can also have that minimisers lie on a straight line or curve: For example for $F(x_1, x_2) = (x_1 - x_2^2)^2$ the minimisers lie on the curve $x_1 = x_2^2$ with $F^* = 0$.



5.2 Global and local minimisers

Consider the function

$$F(x) = \frac{1}{4}x^4 - \frac{1}{12}x^3 - \frac{1}{2}x^2 + \frac{1}{4}x + \frac{1}{2}.$$



This function has a minimum at $x_* = -1$ with value $F_* = \frac{1}{12}$ (orange), but there is another interesting value at $x = 1$ (where $F(x) = 5/12$, green). The point $x = 1$ is clearly not a minimiser of the entire function, but the value at $x = 1$ is less than values nearby. This motivates the idea of a *local minimum*.

Our definitions in (5.1), are **global** (i.e. a global minimiser). By this we mean that we are looking for the value that is the least across all possible values of x. This is a very hard problem! When designing numerical algorithms, we instead often look for **local minimisers**. Later we will see a condition where we can guarantee local minimisers are global minimisers.

Definition 5.2. Scalar domain case: Given a smooth function $F\colon \mathbb{R} \to \mathbb{R}$, we say that a point x^* is a **local minimiser** of F if there exists a value b such that for all $y \in \mathbb{R}$ with $|y - x^*| < b$, $F(x^*) \leq F(y)$.

Vector domain case: Given a smooth function $F\colon \mathbb{R}^n \to \mathbb{R}$, we say that a point $\vec{x}^*$ is a **local minimiser** of F if there exists a value b such that for all $\vec{y} \in \mathbb{R}^n$ with $\|\vec{y} - \vec{x}^*\| < b$, $F(\vec{x}^*) \leq F(\vec{y})$.

5.3 How to determine a local minimiser in the scalar domain case

Looking again at the examples above, we can see that at all minimisers the slope of F is 0 (i.e. the graph is horizontal). We can say this as the derivative of F is zero or $F'(x_*) = 0$. However, the converse is not true: we see points such as the local minimal where the slope is zero but also other points with zero derivative too!

Proposition 5.1. *Let $F\colon \mathbb{R} \to \mathbb{R}$ and let x^* be a local minimiser of F. Then $F'(x^*) = 0$.*

Proof. (*We do not expect you to learn this proof but the result is very important!*)

We see that the definition of local minima says that for h small enough that

$$F(x^* + h) - F(x^*) \geq 0,$$

so

$$\frac{F(x^* + h) - F(x^*)}{h} \geq 0 \qquad \text{if} \quad h \geq 0 \tag{5.2}$$

$$\frac{F(x^* + h) - F(x^*)}{h} \leq 0 \qquad \text{if} \quad h \leq 0. \tag{5.3}$$

If F is smooth, F' exists and the limit

$$F'(x^*) = \lim_{h \to 0} \frac{F(x^* + h) - F(x^*)}{h},$$

has a single value. The results of limits say that in this case (5.2) and (5.3) both hold in the limit too. This is only possible if $F'(x^*) = 0$. □

Example 5.1. Let $F(x) = x^2 - 6x + 12$.

We can compute that $F'(x)$ is given by

$$F'(x) = 2x - 6.$$

If we set $F'(x) = 0$ we can rearrange to see

$$\begin{aligned} F'(x) = 2x - 6 &= 0 \\ 2x &= 6 \\ x &= \frac{6}{2} = 3. \end{aligned}$$

Thus, there is a local minima at $x^* = 3$ with $F^* = F(x^*) = 3^2 - 6 \times 3 + 12 = 3$.

Exercise 5.1. Let $F(x) = \frac{1}{4}x^4 - 2x^3 + \frac{11}{2}x^2 - 6x + 1$. Identify any local minima by finding the derivative of F and finding all points with $F'(x) = 0$.

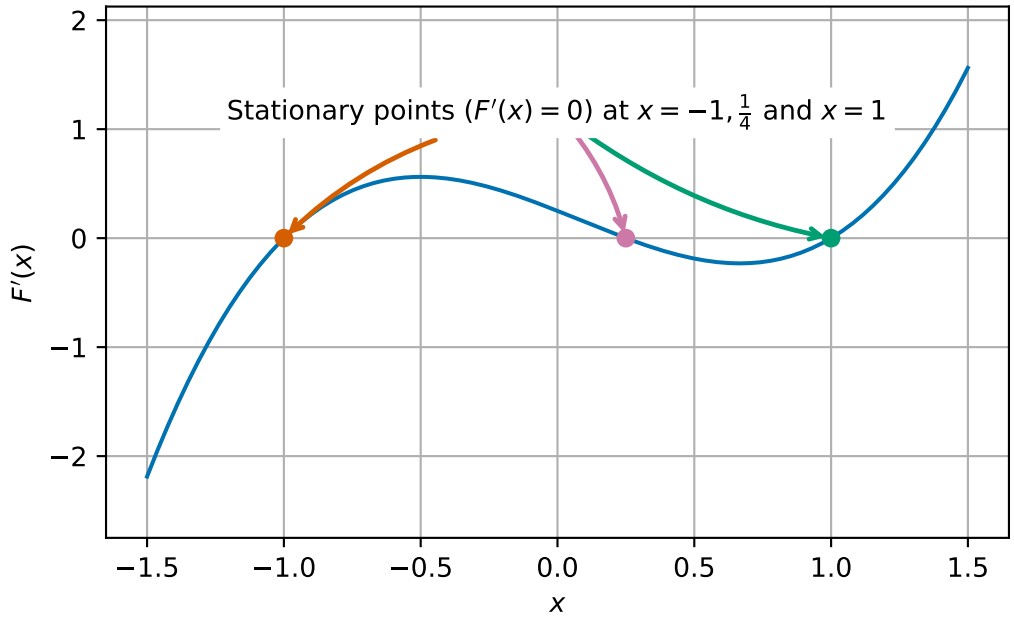
We next revisit the example from the start of this section

$$F(x) = \frac{1}{4}x^4 - \frac{1}{12}x^3 - \frac{1}{2}x^2 + \frac{1}{4}x + \frac{1}{2}.$$

We can compute the derivative as:

$$F'(x) = x^3 - \frac{1}{4}x^2 - x + \frac{1}{4}.$$

Looking at the graph, we can see the F' is zero at three points, $x = -1, 1$ (orange and green) which we have already identified as local minima, but also at $x = \frac{1}{4}$ (purple).



We see that at the local minima (red and green points) the derivative is *increasing* but at the other point (orange), the derivative is decreasing. More formally, we can say that at local minima the second derivative is strictly positive. In this instance, we have

$$F''(x) = 3x^2 - \frac{1}{2}x - 1,$$

so that

$$\begin{aligned}
F''(-1) &= 3 \times (-1)^2 - \frac{1}{2} \times (-1) - 1 = 3 + \frac{1}{2} - 1 = \frac{5}{2} = 2.5 > 0 \\
F''(\tfrac{1}{4}) &= 3 \times (\frac{1}{4})^2 - \frac{1}{2} \times \frac{1}{4} - 1 = \frac{3}{16} - \frac{1}{8} - 1 = -\frac{15}{16} = -0.9375 < 0 \\
F''(1) &= 3 \times 1^2 - \frac{1}{2} \times 1 - 1 = 3 - \frac{1}{2} - 1 = \frac{3}{2} = 1.5 > 0.
\end{aligned}$$

Proposition 5.2. *Let $F \colon \mathbb{R} \to \mathbb{R}$ and $x^* \in \mathbb{R}$ such that $F'(x^*) = 0$ and $F''(x^*) \geq 0$. Then x^* is a local minimiser of F.*

Proof. We only sketch the proof here. The key idea is to use something called Taylor's theorem, which gives us the formula that for x close to x^*, we have that

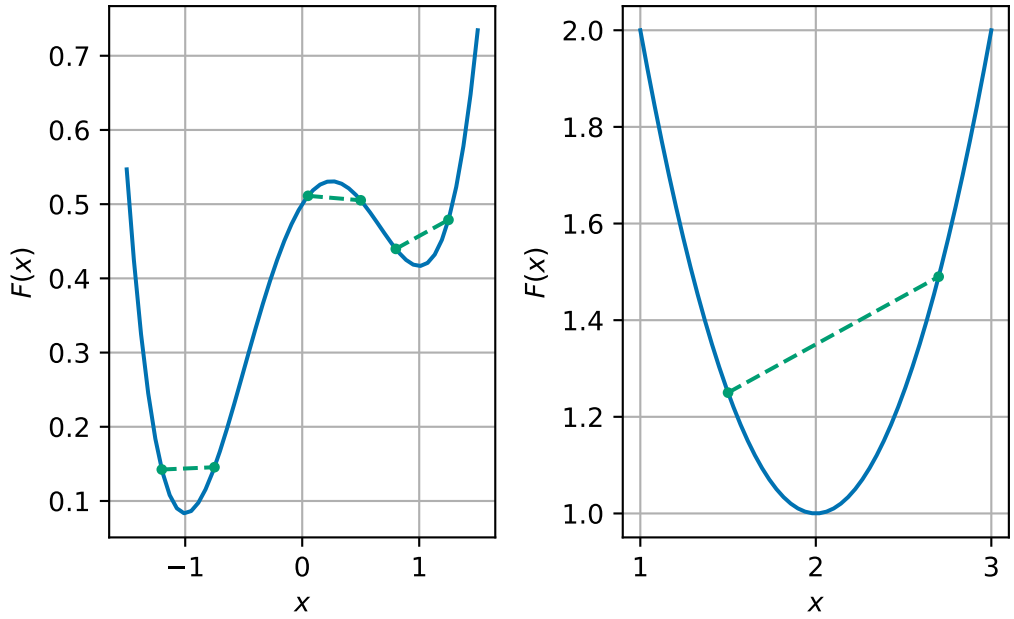
$$F(x) \approx F(x^*) + (x - x^*)F'(x^*) + \frac{1}{2}(x - x^*)^2 F''(x^*).$$

Applying that $F'(x^*) = 0$ and $F''(x^*) \geq 0$, we have

$$F(x) \approx F(x^*) + \frac{1}{2}(x - x^*)^2 F''(x^*) \geq F(x^*).$$

This proof can be made rigorous with the help of further results from mathematical analysis. □

Next, let us compare two examples:



For each example, we have chosen different pairs of points and drawn a straight line between them (green dashed line). We see, in the left hand side example, that sometimes the straight line is above the function (blue curve) and sometimes its below, whereas in the right hand side example, the straight line is above the function, and always will be for any pair of points. This idea is expressed mathematically through the idea of *convexity*.

Definition 5.3. A function $F\colon \mathbb{R} \to \mathbb{R}$ is **locally convex in an interval** $B \subset \mathbb{R}$ if for all points $x, y \in B$:

$$aF(x) + (1 - a)F(y) \geq F(ax + (1 - a)y) \quad \text{for all} \quad a \in [0, 1]. \tag{5.4}$$

We say that F is **convex** if it is convex on $\mathbb{R}$ (i.e., on every interval $B \subset \mathbb{R}$).

It is hard to show convexity with this definition directly. Instead, we normally use the follow equivalent formulations:

Lemma 5.1. *Let $F\colon \mathbb{R} \to \mathbb{R}$ be a smooth function. Then the following are equivalent:*

1. *F is locally convex on an interval B*
2. *The derivative of F satisfies*

$$F(x) \geq F(y) + F'(y)(x-y) \quad \text{for all} \quad x, y \in B. \tag{5.5}$$

3. *The second derivative of F is positive in B: $F''(x) \geq 0$ for all $x \in B$.*

Remark 5.1. Condition 2 can be expressed informally as saying that the graph of F lies above its tangent.

Condition 3 links directly to the idea of linking local and global minima through Proposition 5.2.

Proof. [don't need to know this one either]

We prove (1) is equivalent to (2) and (2) is equivalent to (3).

(1) $\Rightarrow$ (2).

Assume F is (locally) convex on B. Fix $x, y \in B$ and $a \in (0, 1]$. By convexity,

$$aF(x) + (1-a)F(y) \;\geq\; F(ay + (1-a)x) = F(y + a(x-y)).$$

Rearranging gives

$$F(x) \;\geq\; \frac{F(y + a(x-y)) - F(y)}{a} + F(y) \;=\; (x-y)\,\frac{F(y+h) - F(y)}{h} + F(y),$$

where we set $h := a(x-y)$. As $a \downarrow 0$ we have $h \to 0$ (with the same sign as $x-y$). Since F is smooth, the difference quotient tends to $F'(y)$ regardless of the sign of h. Thus,

$$F(x) \;\geq\; F(y) + F'(y)(x-y),$$

which is (5.5).

(2) $\Rightarrow$ (1).

We start with $x, y \in B$ and let $z = ax + (1-a)y$, then we have

$$F(x) \geq F(z) + F'(z)(x-z) \quad \text{and} \quad F(y) \geq F(z) + F'(z)(y-z).$$

Multiplying the first inequality by a and the second by $1-a$ and adding gives

$$aF(x) + (1-a)F(y) \geq F(z) + F'(z)(a(x-z) + (1-a)(y-z)).$$

But from the definition of z, we have

$$a(x-z)+(1-a)(y-z)=ax+(1-a)y-z=0.$$

Hence, we can infer that

$$aF(x)+(1-a)F(y)\geq F(z)=F(ax+(1-a)y).$$

(2) $\Rightarrow$ (3).

We can apply (5.5) to see that for any $x,y\in B$, we have

$$F(x)\geq F(y)+F'(y)(x-y)$$
$$F(y)\geq F(x)+F'(x)(y-x).$$

Adding these inequality and rearranging gives

$$(F'(y)-F'(x))(y-x)\geq 0.$$

This inequality says that F' is nondecreasing on B. If we let $y=x+h$ for $h>0$, we have

$$\frac{F'(x+h)-F'(x)}{h}\geq 0.$$

Taking the limit $h\to 0$ shows that $F''(x)\geq 0$.

(3) $\Rightarrow$ (2).

This result is a little hard and relies on a result from mathematical analysis (the Mean Value Theorem) which is not covered by this course. The key idea is to show that $F''(x)\geq 0$ implies that F' is nondecreasing and then if F' is nondecreasing that (5.5) holds. □

Example 5.2. We see that condition 3 in Lemma 5.1 is often the easiest to show.

1. Take for example $F(x)=x^2-4x+5$. Then

   $$F'(x)=2x-4 \quad \text{and} \quad F''(x)=2>0.$$

   Hence we can see that F is convex.

   This example, is somewhat of a prototypical example. It is often helpful when thinking of convex functions to think of something like a quadratic function with positive leading coefficient.

2. (hard) Take $F(x)=\frac{1}{4}x^4-\frac{1}{12}x^3-\frac{1}{2}x^2+\frac{1}{4}x+\frac{1}{2}$, then we can factor F'' as

   $$F''(x)=3(x-\tfrac{2}{3})(x+\tfrac{1}{2}).$$

   We can see that F'' is negative if one of the factors on the right hand side is negative. That happens when $-\frac{1}{2}<x<\frac{2}{3}$. Hence, we can infer that F is not globally convex.

Exercise 5.2. Show that the following functions are or are not convex on $\mathbb{R}$: 1. $F(x) = x^2 - 6x + 12$ 2. $F(x) = \frac{1}{4}x^4 - 2x^3 + \frac{11}{2}x^2 - 6x + 1$

Proposition 5.3. *Let $F : \mathbb{R} \to \mathbb{R}$ and x^* be a point such that $F'(x^*) = 0$. If F is locally convex in a region B with $x^* \in B$ then x^* is a local minimiser of F.*

Proof. (*We do not expect you to learn this proof but the result is very important!*)

Let $y \in B$, then from Lemma 5.1, we have

$$F(y) \geq F(x^*) + F'(x^*)(y - x^*).$$

Since $F'(x^*) = 0$, we see that

$$F(y) \geq F(x^*) + F'(x^*)(y - x^*) = F(x^*) + 0(y - x^*) = F(x^*).$$

Hence we can see that x^* is a local minimiser in the interval B. □

Theorem 5.1. *Let $F : \mathbb{R} \to \mathbb{R}$ be convex. Then any local minimiser is a global minimiser.*

Proof. Let x^* be a local minimiser of F, then $F'(x^*) = 0$ (from Proposition 5.1). Applying **?@lem-convexity-diff**, we see that for any y, we have

$$F(y) \geq F(x^*) + F'(x^*)(y - x^*) = F(x^*).$$

Hence x^* is in fact a global minimiser. □

Example 5.3. Take the same example as before $F(x) = x^2 - 4x + 5$. Then

$$F'(x) = 2x - 4.$$

We can easily see that $F'(2) = 0$ hence $x^* = 2$ is a local and indeed a global minimiser since F is convex.

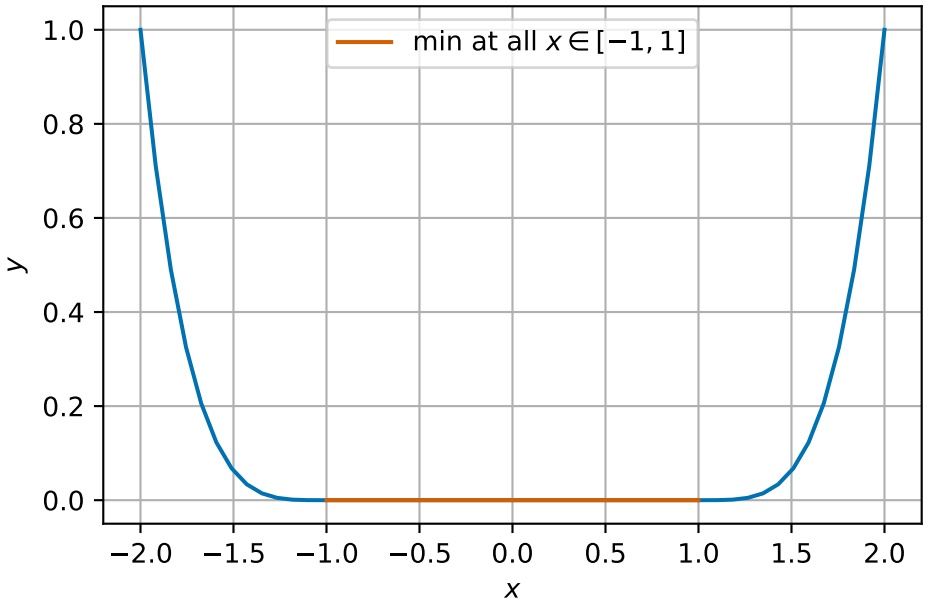
We can also see this by completing the square:

$$F(x) = x^2 - 4x + 5 = (x - 2)^2 + 1.$$

We can read off that F has a minimum at the same point that $(x - 2)^2 = 0$, i.e. that $x^* = 2$.

Remark 5.2. Minimisers of convex functions are not necessarily unique. Consider $F\colon \mathbb{R} \to \mathbb{R}$ given by

$$F(x) = \begin{cases} (x+1)^4 & \text{for} \quad x \leq -1 \\ 0 & \text{for} \quad -1 < x < 1 \\ (x-1)^4 & \text{for} \quad x \geq 1. \end{cases}$$



F is a convex, smooth function, but all points $x \in [-1, 1]$ are minimisers.

5.4 Summary

Optimisation is a wide area of numerical computation with many different application areas. In this module, we will only consider continuous optimisation problems. We have provided several examples of how different types of minima can occur in one and two dimensions using plots to identify minima. In one dimension, we have provided a characterisation of local minima and how to link them to global minima using the idea convexity.

6 Root finding methods for one dimensional optimisation

In one dimension, we will proceed with a simple idea: to find candidate minima of functions $F\colon \mathbb{R} \to \mathbb{R}$, we will find points at which the derivative of F is 0. To help us with notation, in this section we will write $F' = f$.

> Given a smooth function $f(x)$, the problem is to find a point x^* such that $f(x^*) = 0$. That is, x^* is a solution of the equation $f(x) = 0$ and is called a **root of** $f(x)$.

Remark 6.1. In one dimension, a local minimum occurs at point where $F'(x) = 0$, but not every root of F' is a minimum. The focus on this lecture is finding possible minima.

Example 6.1.

1. The linear equation $f(x) = ax + b = 0$ has a single solution at $x^* = \frac{-b}{a}$.
2. The quadratic equation $f(x) = ax^2 + bx + c = 0$ is nonlinear, but simple enough to have a known formula for the solutions:
$$x^* = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$
3. A general nonlinear equation $f(x) = 0$ rarely has a formula like those above which can be used to calculate its roots.

It is also sometimes better to use numerical methods to solve an equation even if an exact formula exists:

- the exact formula may have difficulties due to rounding errors;
- the exact formula may be more expensive to compute.

Example 6.2. We want to solve the quadratic equation
$$f(x) = x^2 - (m + \frac{1}{m})x + 1 = 0,$$
for large values of m. The exact solutions are m and $\frac{1}{m}$.

```python
# implementation of quadratic formula
def quad(a, b, c):
    """
    Return the roots of the quadratic polynomial a x^2 + b x + c = 0.
    """
    d = np.sqrt(b * b - 4 * a * c)
    return (-b + d) / (2 * a), (-b - d) / (2 * a)


# test for m = 1e8
m = 1e8
roots = quad(1, -(m + 1 / m), 1)

print(f"roots {roots[0]:.4e} and {roots[1]:.4e}")
```

```
roots 1.0000e+08 and 1.4901e-08
```

These values look reasonable, but let's check the errors:

```python
# absolute and relative errors
exact_roots = (m, 1 / m)

abs_error = (abs(roots[0] - exact_roots[0]), abs(roots[1] - exact_roots[1]))
rel_error = (abs_error[0] / m, abs_error[1] / (1 / m))

print(f"abs errors {abs_error[0]:.4e} {abs_error[1]:.4e}")
print(f"rel errors {rel_error[0]:.4e} {rel_error[1]:.4e}")
```

```
abs errors 0.0000e+00 4.9012e-09
rel errors 0.0000e+00 4.9012e-01
```

The absolute errors are quite small, but the relative error for the second root is huge - almost 50%!

6.1 Example problems

We will test our methods on the following problems.

Example 6.3 (Square root example)**.** We want to find the value of the square root of 2. We can solve this by finding the (positive) root of

$$f(x) = x^2 - 2.$$

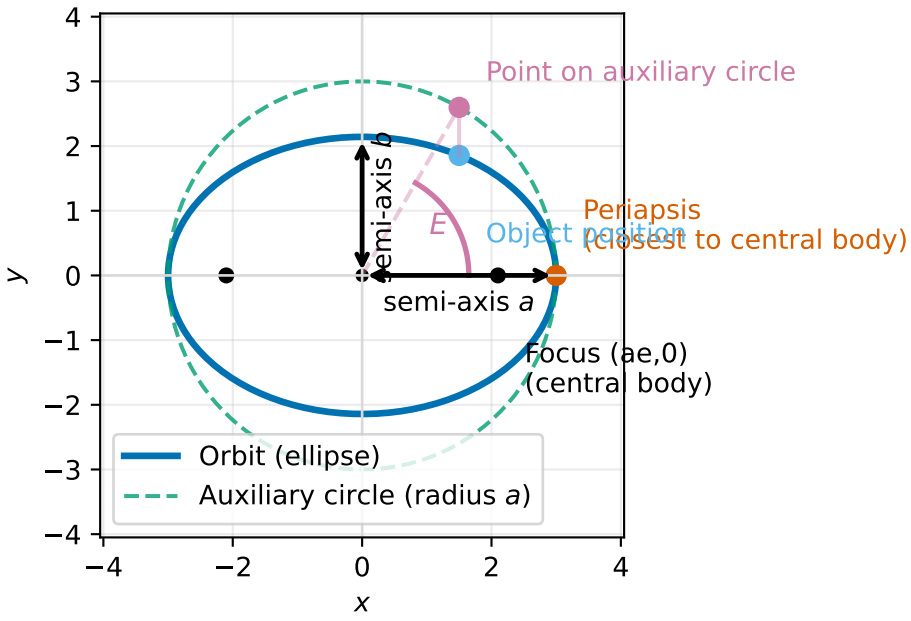
This problem is interesting in that we know there are two solutions, but we only want the positive one. Our algorithms should take that into account.

Example 6.4 (Kepler's equation, elliptic orbits)**.** When an object moves under gravity around a planet or star, its path is often well-approximated by an ellipse. To work out where the object is at a given time, we use a standard relationship called Kepler's equation. The key point here is: it reduces to finding a root of a function $f(E) = 0$.

An ellipse has two semi-axes a and b and we assume that $a > b$. We denote by $e = \sqrt{1 - \frac{b^2}{a^2}}$ the eccentricity of the ellipse. The closest point on the ellipse to the central body (i.e. planet or star, which is at one of the foci of the ellipse) of the object's trajectory is called the periapsis.

To connect time to position, it's convenient to use an angle-like variable that increases at a constant rate - this variable is called the mean anomaly, M (measured in radians). $M = 0$ corresponds to periapsis and $M = \pi$ corresponds to half an orbit later.

There is a helpful intermediate angle called the eccentric anomaly which we write as E. Since M is not the actual geometric angle around the ellipse, we use E as a useful angle from which we can compute quantities such that the coordinates of the object.



For an elliptic orbit with eccentricity $0 \leq e < 1$, the mean anomaly M is related to the eccentric anomaly E by Kepler's equation:

$$E - e\sin(E) = M.$$

The problem is: Given e (eccentricity) and M (mean anomaly at a given time, in radians), find E^* (eccentric anomaly). We can write this as a root finding problem with

$$f(E) = E - e\sin(E) - M.$$

We should be careful to find a solution only in $[0, 2\pi)$. In our tests we will use $e = 0.7, M = 2.0$. Note that

$$f'(E) = 1 - e\cos E \geq 1 - e > 0,$$

so f is strictly increasing and we know we only have one solution.

6.2 Iterative methods

All our methods will use the idea of **iteration** (or **iterative improvement**). We saw this idea when we were using Jacobi and Gauss-Seidel iteration to solve systems of linear equations.

- Given an initial estimate of the root $x^{(0)}$, try to generate a better approximate $x^{(1)}$ of the root.
- Once $x^{(1)}$ has been computed, it can be used to compute another estimate $x^{(2)}$ in the same way (and so on…).
- Generally, $x^{(i)}$ is used to compute $x^{(i+1)}$ (although other previous estimates may be used as well).
- There are many methods for doing this!

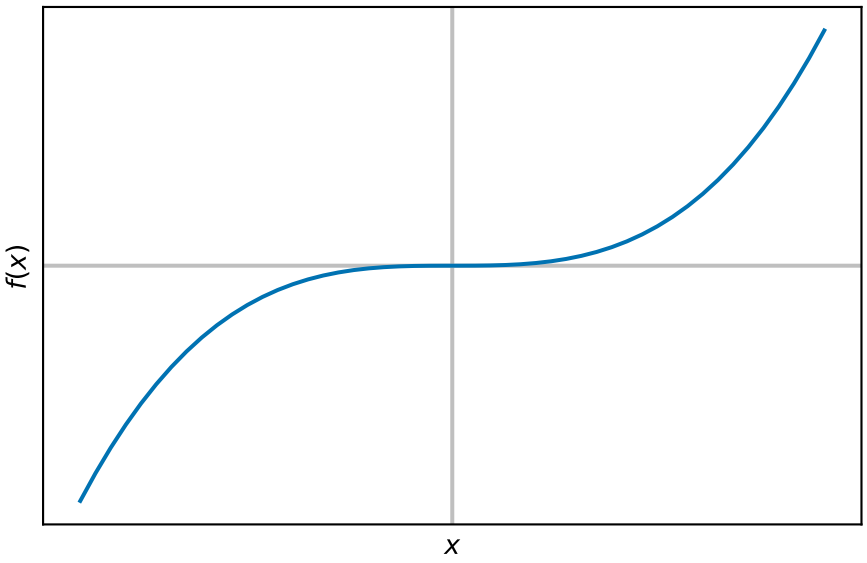
There are a few issues we should consider with iterative methods:

1. Does the iteration converge?
2. In other words, will $x^{(i+1)}$ be a better estimate than $x^{(i)}$? In what sense – e.g. $|f(x^{(i)})|$ smaller or $|x^{(i)} - x^*|$ smaller?
3. Is there a way to check this as part of the computation? Can we prove this?
4. If the iteration does converge, then how quickly does this happen?
5. How should we decide when to stop the iteration?

Remark 6.2. We briefly comment that iteration is *different* to the idea of time stepping we met when solving differential equations. The number of time steps is a given parameter when solving a differential equation but we (normally) do not know how many iterations of a root finder are needed to find a solution (for a particular stopping criterion).

6.3 Bisection method

The first method that we introduce is the bisection method. This is a simple, slow but very robust method. It's based on a simple idea: if a function is smooth (continuous) on an interval and changes sign between the two end points (either positive to negative or negative to positive), then it must be zero some point in between. (There is a mathematical analysis theorem called the Intermediate Value Theorem that gives us this result).



6.3.1 The algorithm

The algorithm steps are:

1. Suppose we *know* there is a root x^* (i.e with $f(x^*) = 0$) between two end points a and b (we sometimes call $[a, b]$ the *bracket*).
2. Call $c = \frac{a+b}{2}$, then the solution is either between a and c or between c and b.
3. If the solution is between a and c:
    - then go to step 1 with the end points as a and c,
    - otherwise, go to step 1 with the end points c and b.

We repeat this process until $b - a < TOL$ where TOL is a user-supplied value.

We can check if the root is between a and b by checking if the signs of $f(a)$ and $f(b)$ are different: i.e. $f(a)f(b) < 0$.

6.3.2 Convergence analysis

Assume that k steps have been taken from an initial bracket (a, b). We notice that at each step, the bracket size is halved. This means after k steps, the length of the interval will be $\frac{b-a}{2^k}$.

For large initial bracket size (i.e. large $b - a$) or small TOL (high accuracy requirement) k will be large.

6.3.3 Examples

$$f(x) = x^2 - 2.$$

```
it  a        b        f(a)        f(b)
 1 1.000000 1.500000 -1.0000e+00 2.5000e-01
 2 1.250000 1.500000 -4.3750e-01 2.5000e-01
 3 1.375000 1.500000 -1.0938e-01 2.5000e-01
 4 1.375000 1.437500 -1.0938e-01 6.6406e-02
 5 1.406250 1.437500 -2.2461e-02 6.6406e-02
 6 1.406250 1.421875 -2.2461e-02 2.1729e-02
 7 1.414062 1.421875 -4.2725e-04 2.1729e-02
 8 1.414062 1.417969 -4.2725e-04 1.0635e-02
 9 1.414062 1.416016 -4.2725e-04 5.1003e-03
10 1.414062 1.415039 -4.2725e-04 2.3355e-03
11 1.414062 1.414551 -4.2725e-04 9.5391e-04
12 1.414062 1.414307 -4.2725e-04 2.6327e-04
13 1.414185 1.414307 -8.2001e-05 2.6327e-04
14 1.414185 1.414246 -8.2001e-05 9.0633e-05

1.414215087890625
```

We see that it takes 14 iterations to find the solution to a tolerance of 10^{-4} for the square root problem.

$$f(x) = x - 0.7\sin(x) - 2.0$$

```
it  a        b        f(a)        f(b)
 1 0.000000 3.141593 -2.0000e+00 1.1416e+00
 2 1.570796 3.141593 -1.1292e+00 1.1416e+00
 3 2.356194 3.141593 -1.3878e-01 1.1416e+00
 4 2.356194 2.748894 -1.3878e-01 4.8102e-01
 5 2.356194 2.552544 -1.3878e-01 1.6364e-01
```

```
 6 2.356194 2.454369 -1.3878e-01 1.0294e-02
 7 2.405282 2.454369 -6.4809e-02 1.0294e-02
 8 2.429826 2.454369 -2.7395e-02 1.0294e-02
 9 2.442097 2.454369 -8.5847e-03 1.0294e-02
10 2.442097 2.448233 -8.5847e-03 8.4623e-04
11 2.445165 2.448233 -3.8713e-03 8.4623e-04
12 2.446699 2.448233 -1.5131e-03 8.4623e-04
13 2.447466 2.448233 -3.3356e-04 8.4623e-04
14 2.447466 2.447850 -3.3356e-04 2.5630e-04
15 2.447658 2.447850 -3.8638e-05 2.5630e-04
16 2.447658 2.447754 -3.8638e-05 1.0883e-04

2.4477060315696475
```

We see that it takes 16 iterations to find the solution to a tolerance of 10^{-4} for the Kepler problem

Exercise 6.1. For the square root problem, for which of the following brackets do you expect the algorithm to find a solution? For cases where you expect the algorithm to find a solution, how many iterations do you expect for a tolerance of 10^{-4}?

1. $(1.0, 1.5)$
2. $(0.0, 4.0)$
3. $(-4.0, 4.0)$
4. $(-2.0, 0.0)$

6.3.4 Weaknesses of the bisection algorithm

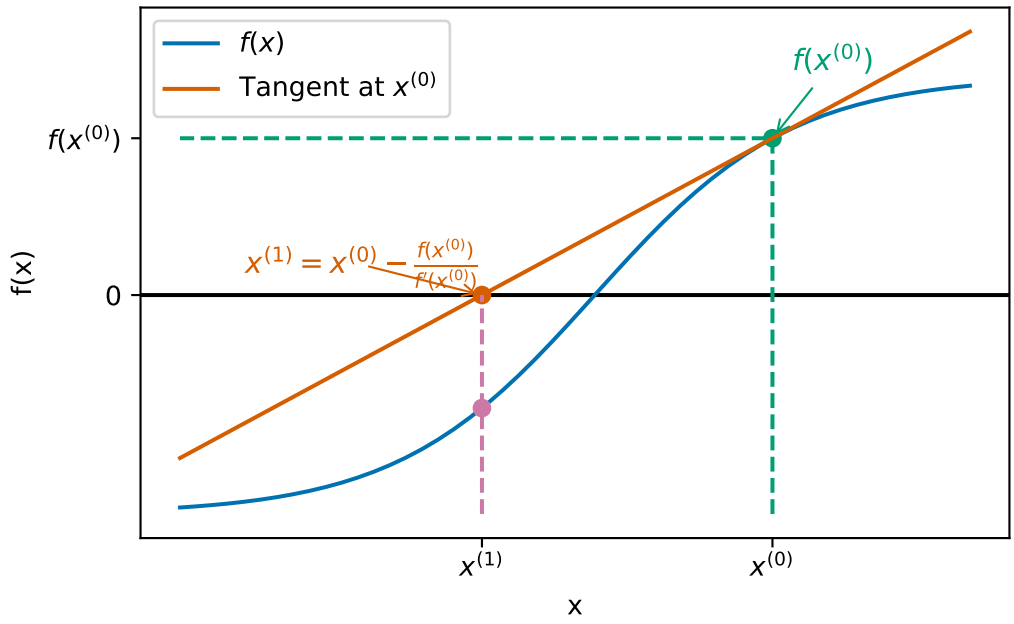
Bisection is a reliable method for finding solutions of $f(x) = 0$ provided an initial bracket can be found. It is far from perfect however:

- finding an initial bracket is not always easy;
- it can take a very large number of iterations to obtain an accurate answer;
- it can never find a solution of $f(x)$ for which f does not change sign (e.g. $f(x) = (x-1)^2$).

6.4 Newton's method

Newton's method (or the Newton-Raphson iteration) is an alternative algorithm for solving $f(x) = 0$. On the positive side, it does not need an initial bracket (just one initial value), it converges much more quickly than bisection and it can solve problems where the solution is

also a turning point. On the negative side, it is not guaranteed to converge (unlike bisection with a good initial bracket).



We start at our initial guess $x^{(0)}$ and find the corresponding point on the graph of the function $(x^{(0)}, f(x^{(0)}))$. We find the tangent to the graph at $(x^{(0)}, f(x^{(0)}))$ and follow this line to the x-axis. Where the tangent line intersects the x-axis is our new value for $x^{(1)}$.

In the picture, we can see two ways to calculate the tangent:

1. We can use the derivative of f to see that the slope of the tangent is $f'(x^{(0)})$.
2. We can see that we have a right-angled triangle with vertices at $(x^{(1)}, 0)$, $(x^{(0)}, 0)$ and $(x^{(0)}, f(x^{(0)})$ which has slope

$$\frac{\text{change in } y}{\text{change in } x} = \frac{f(x^{(0)}) - 0}{x^{(0)} - x^{(1)}}.$$

Equating the two slopes, we see that

$$f'(x^{(0)}) = \frac{f(x^{(0)})}{x^{(0)} - x^{(1)}},$$

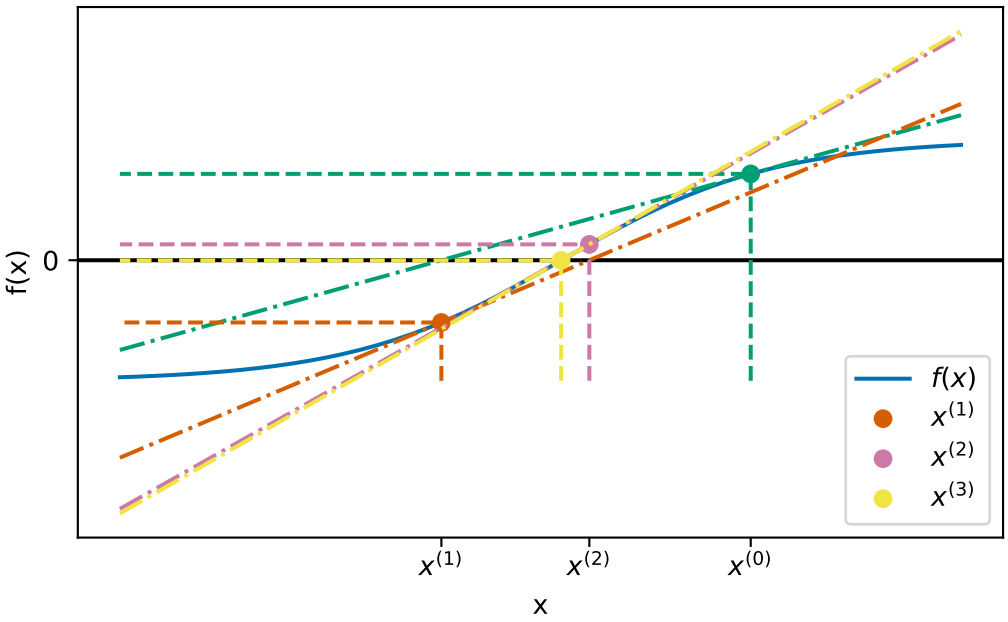
which we can rearrange to

$$x^{(1)} = x^{(0)} - \frac{f(x^{(0)})}{f'(x^{(0)})}.$$

This is the formula for a single Newton update!

Once we have our updated estimate $x^{(1)}$, we can continue the iteration:

$$x^{(i+1)} = x^{(i)} - \frac{f(x^{(i)})}{f'(x^{(i)})}.$$



This is quite a messy plot but it appears the Newton's method converges very quickly!

Remark 6.3.

1. The iteration formula requires an initial guess at the solution $x^{(0)}$.
2. In order to apply Newton's method, we need to be able to compute an expression for the derivative of $f(x)$ - this may not always be possible or easy.
3. You will see a variant of Newton's method that allows the derivative to be approximated in the lab session.
4. We typically use the absolute value of $f(x^{(i)})$ as the variable we compare against a tolerance in a stopping criteria.

6.4.1 Examples

$$f(x) = x^2 - 2.$$

```
it  x         f(x)         df(x)
 1 1.500000 2.5000e-01 3.0000
```

```
 2 1.416667 6.9444e-03 2.8333
 3 1.414216 6.0073e-06 2.8284


1.4142156862745099
```

We see that it takes *only* 3 iterations to find the solution to a tolerance of 10^{-4} for the square root problem.

$$f(x) = x - 0.7\sin(x) - 2.0$$

```
it  x         f(x)        df(x)
 1 2.470068 3.4541e-02 1.5480
 2 2.447754 1.0943e-04 1.5382
 3 2.447683 1.1329e-09 1.5381


np.float64(2.447683215352491)
```

We see that it takes 3 iterations to find the solution to a tolerance of 10^{-4} for the Kepler problem

Example 6.5. Consider $f(x) = x^2 - 5$ and $x^{(0)} = 2$. This gives $f'(x) = 2x$ so the iterative formula is:

$$x^{(i+1)} = x^{(i)} - \frac{(x^{(i)})^2 - 5}{2x^{(i)}} = \frac{(x^{(i)})^2 + 5}{2x^{(i)}}.$$

The iteration then proceeds as:

$$x^{(1)} = \frac{(x^{(0)})^2 + 5}{2x^{(0)}} = \frac{2^2 + 5}{2 \times 2} = \frac{9}{4} = 2.25.$$
$$x^{(2)} = \frac{(x^{(1)})^2 + 5}{2x^{(1)}} = \frac{(2.25)^2 + 5}{2 \times 2.25} = \frac{161}{72} \approx 2.23611.$$

Note that $\sqrt{5} = 2.23607\ldots$.

Exercise 6.2. Write down an expression for the Newton iteration formula for the following functions and then carry out 2 iterations using the resulting iteration and given initial value $x^{(0)}$.

1. $f(x) = x^3 - 2x - 5$ with $x^{(0)} = 2$.
2. $f(x) = x^3 - 2$ with $x^{(0)} = 1$.

6.4.2 Challenges for Newton's method

The first obvious challenge for Newton's method comes from the idea that we divide by something that might be zero in our update formula – what if $f'(x^{(i)}) = 0$? In this case, there isn't much we can do and Newton's method will fail.

Example 6.6. Consider applying Newton's method to the function $f(x) = x^3 + 2x^2 + x + 1$ with $x^{(0)} = 0$. This gives $f'(x) = 3x^2 + 4x + 1$. Starting from $x^{(0)} = 0$, we get:

$$x^{(1)} = x^{(0)} - \frac{f(x^{(0)})}{f'(x^{(0)})} = 0 - \frac{0^3 + 2 \times 0 + 0 + 1}{3 \times 0^2 + 4 \times 0 + 1} = 0 - \frac{1}{1} = -1.$$

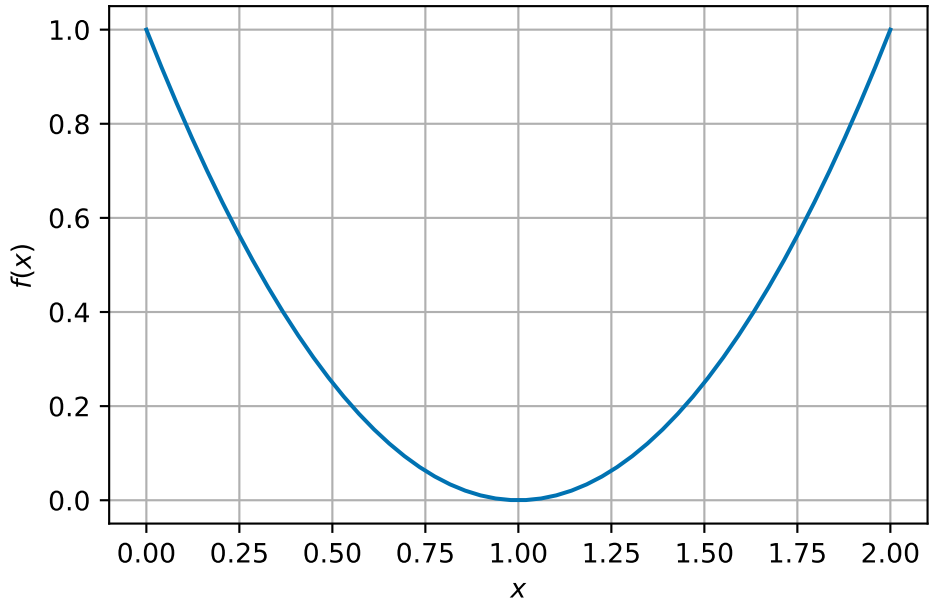
But we see that

$$f'(x^{(1)}) = 3 \times (-1)^2 + 4 \times -1 + 1 = 0.$$

So the iteration cannot proceed.

Another similar case is what if the derivative at the root is zero (i.e., $f'(x^*) = 0$), in this case the iteration can still proceed but the iteration is *slower*.

Consider $f(x) = (x-1)^2 = x^2 - 2x + 1$:



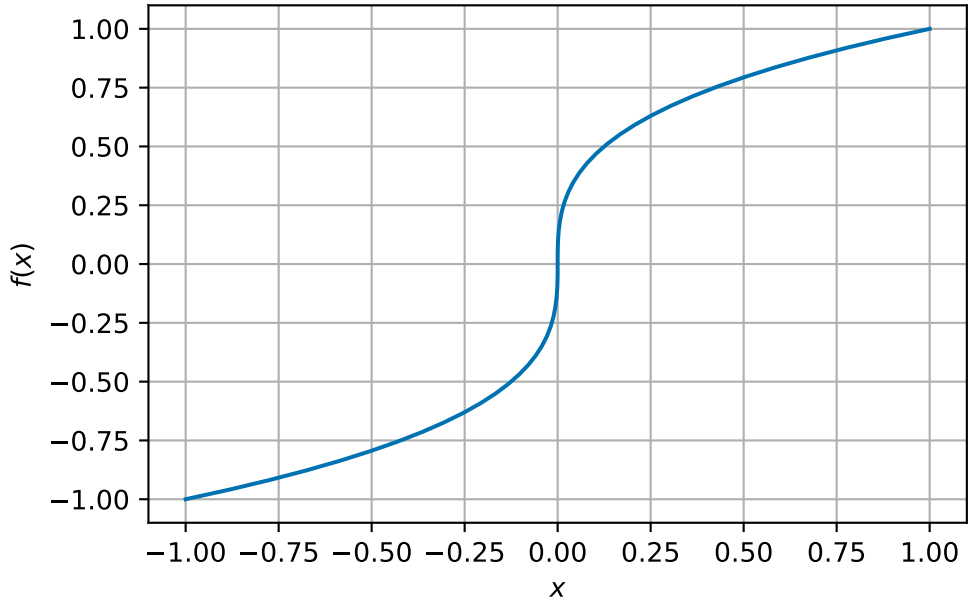

```
it  x         f(x)        df(x)
 1 1.500000 2.5000e-01 1.0000
 2 1.250000 6.2500e-02 0.5000
```

```
 3 1.125000 1.5625e-02 0.2500
 4 1.062500 3.9062e-03 0.1250
 5 1.031250 9.7656e-04 0.0625
 6 1.015625 2.4414e-04 0.0312
 7 1.007812 6.1035e-05 0.0156

1.0078125
```

It takes 7 iterations to find the root in this case (more than the other Newton examples). We note here also that the convergence criterion is $|f(x^{(i)})| < TOL$ which does not guarantee that $|x^* - x^{(i)}| < TOL$!

There are also cases where Newton's method does not converge even when a root exists!

Example 6.7. Consider the function $f(x) = x^{1/3}$. Clearly f has a root at $x^* = 0$ ($f(0) = 0$).



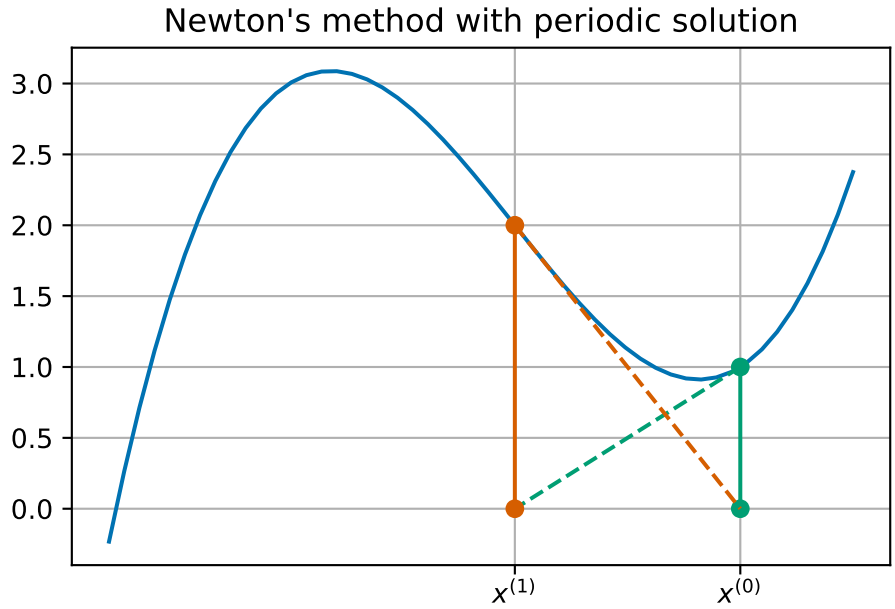
The derivative is $f'(x) = \frac{1}{3}x^{-2/3}$ and the Newton iteration is

$$x^{(i+1)} = x^{(i)} - \frac{(x^{(i)})^{1/3}}{\frac{1}{3}(x^{(i)})^{-2/3}} = x^{(i)} - 3x^{(i)} = -2x^{(i)}.$$

We can see that the general term in this sequence is $x^{(i)} = (-2)^i x^{(0)}$ – this is clearly not going to converge to 0!

It is even possible for the iteration to simply cycle between two values repeatedly.

Example 6.8. Consider the function $f(x) = x^3 - 2x + 2$. This has derivative $f'(x) = 3x^2 - 2$.



Starting an iteration at $x^{(0)} = 1$ gives:

```
it  x         f(x)        df(x)
 1 0.000000 2.0000e+00 -2.0000
 2 1.000000 1.0000e+00 1.0000
 3 0.000000 2.0000e+00 -2.0000
 4 1.000000 1.0000e+00 1.0000
 5 0.000000 2.0000e+00 -2.0000
 6 1.000000 1.0000e+00 1.0000
 7 0.000000 2.0000e+00 -2.0000
 8 1.000000 1.0000e+00 1.0000
 9 0.000000 2.0000e+00 -2.0000
10 1.000000 1.0000e+00 1.0000
11 0.000000 2.0000e+00 -2.0000
12 1.000000 1.0000e+00 1.0000
13 0.000000 2.0000e+00 -2.0000
14 1.000000 1.0000e+00 1.0000
15 0.000000 2.0000e+00 -2.0000
16 1.000000 1.0000e+00 1.0000
17 0.000000 2.0000e+00 -2.0000
18 1.000000 1.0000e+00 1.0000
19 0.000000 2.0000e+00 -2.0000
```

```
20 1.000000 1.0000e+00 1.0000

1.0
```

6.5 Summary

We have presented two methods for finding roots of nonlinear equations: the bisection method and Newton's method. We summarise their properties in the following table:

	Bisection	Newton
Simple algorithm	yes	yes
Starting values	bracket	one
Iterations	lots	normally fewer
Convergence	with good bracket	not always
Turning point roots	no	slower
Use derivative	no	yes

There are many more methods available - you will see one on the lab exercise sheet.

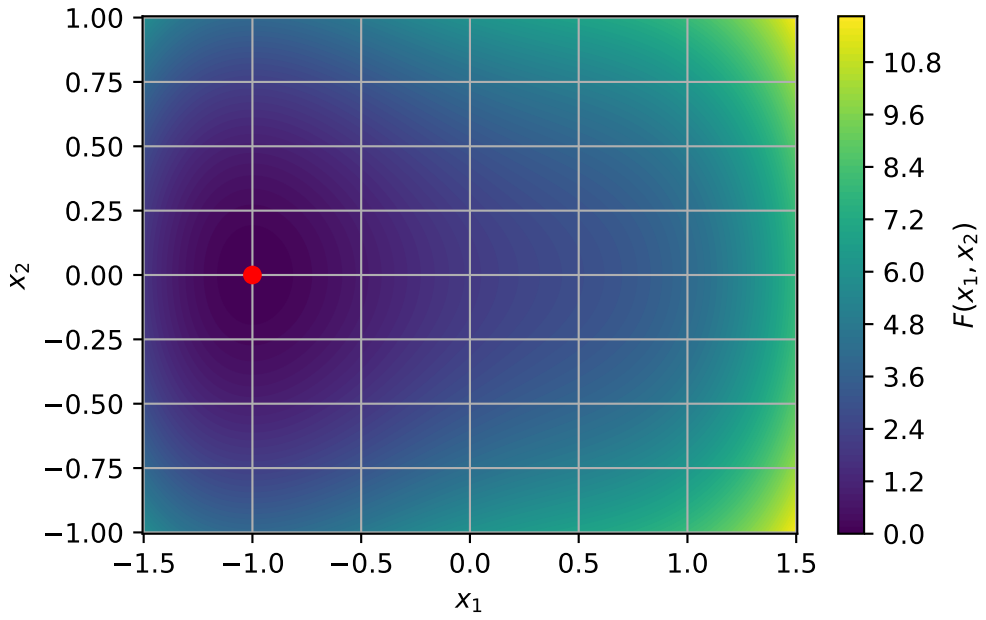
7 What about functions with higher dimension domains?

When we were working with function $F\colon \mathbb{R} \to \mathbb{R}$, we had the key idea to find the points where the derivative of F is 0. We have developed theory to tell us when these points are local or global minima and some methods to find these points.

> When working with functions with two dimensional (or more) domains, how do we compute derivatives?

Throughout this section we will work with the example function:

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2.$$



Some how we want to have the idea that if you move away from the minimum in *any* direction then the function increases. Mathematically, we could write this as saying: Given a direction $\vec{d} \in \mathbb{R}^n$, a point $\vec{x}^*$ is a local minimum if

$$F(\vec{x}^* + h\vec{d}) \geq F(\vec{x}^*) \quad \text{for all small} \quad h.$$

We could try to use our ideas from one dimension then by considering the function $D_{\vec{d}}\mathbb{R} \to \mathbb{R}$ given by

$$D_{\vec{d}}(h) = F(\vec{x}^* + h\vec{d}).$$

This is a well defined one-dimensional function so we can apply all our theory as before.

TODO I think we need the directional derivative at $h = 0$.

Example 7.1. Consider the function

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2$$

and the directions $\vec{d} = (1, 0)^T, (0, 1)^T$ and $(1, 1)^T$. Find $D_{\vec{d}}(h)$ and $D'_{\vec{d}}(h)$ at $\vec{x}^* = (-1, 0)$.

1. For $\vec{d} = (1, 0)^T$, we have

$$\begin{aligned} D_{\vec{d}}(h) &= F(-1 + h, 0) \\ &= (-1 + h)^4 - (-1 + h)^2 + 2(-1 + h) + 4 \times 0^2 + 2 \\ &= (-1)^4 + 4 \times (-1)^3 \times h + 6 \times (-1)^2 \times h^2 + 4 \times (-1) \times h^3 + h^4 - (-1)^2 - 2 \times (-1) \times h - h^2 + 2 \\ &= 1 - 4h + 6h^2 - 4h^3 + h^4 - 1 + 2h - h^2 - 2 + 2h + 2 \\ &= h^4 - 4h^3 + 5h^2, \end{aligned}$$

   so

$$D'_{\vec{d}}(h) = 4h^3 - 12h^2 + 10h.$$

2. For $\vec{d} = (0, 1)^T$, we have

$$\begin{aligned} D_{\vec{d}}(h) &= F(-1, 0 + h) \\ &= (-1)^4 - (-1)^2 + 2 \times (-1) + 4h^2 + 2 \\ &= 1 - 1 - 2 + 4h^2 + 2 = 4h^2, \end{aligned}$$

   so

$$D'_{\vec{d}}(h) = 8h.$$

3. For $\vec{d} = (1, 1)^T$, we have

$$\begin{aligned} D_{\vec{d}}(h) &= F(-1 + h, 0 + h) \\ &= (-1 + h)^4 - (-1 + h)^2 + 2(-1 + h) + 4 \times h^2 + 2 \\ &= (h^4 - 4h^3 + 5h^2) + (4h^2) \\ &= h^4 - 4h^3 + 9h^2, \end{aligned}$$

   so

$$D'_{\vec{d}}(h) = 4h - 12h^2 + 18h.$$

We note that each derivatives depends on h and that for each derivative we have $D'_{\vec{d}}(0) = 0$.

The object we have been computing in this example is called the *directional derivative*:

Definition 7.1. Let $F\colon \mathbb{R}^n \to \mathbb{R}$. The **direction derivative of F at $\vec{x}$ in a direction $\vec{d}$** is given by $D'_{\vec{d}}(h)$.

There is something slightly strange about how directional derivatives depend on both the direction and the size of $\vec{d}$. However this is simply part of the definition and something we should be aware of when doing this type of computation.

So we might want to think about building theory based on directional derivatives. However, as we have seen in the picture of F above, there are still lots of directions to check. We have checked 3 different directions but there are still infinitely many to do!

There is something else we can do, and the calculations for the direction $\vec{d} = (1,1)^T$ give us a clue! We saw that

$$D'_{(1,1)^T}(h) = D'_{(1,0)^T}(h) + D'_{(0,1)^T}(h).$$

This is also true in general:

$$D'_{\vec{d}^{(1)}+\vec{d}^{(2)}}(h) = D'_{\vec{d}^{(1)}}(h) + D_{\vec{d}^{(2)}}(h)$$
$$D'_{w\vec{d}}(h) = wD'_{\vec{d}}(h).$$

So if we can find a basis (remember that term from semester 1?!?), we can write any directional derivative in terms of the basis vectors. We can make a simple choice for this: the coordinate axes!

Let $\vec{e}^{(j)}$ be the vector with 1 in the jth place and 0 in every other place for $j = 1, 2, \dots n$. For example in $\mathbb{R}^2$, we have $\vec{e}^{(1)} = (1,0)^T$ and $\vec{e}^{(2)} = (0,1)^T$. These form a basis. So any direction $\vec{d}$ can be written as

$$\vec{d} = (d_1, d_2, \dots, d_n) = \sum_{j=1}^{n} d_j \vec{e}^{(j)},$$

so that

$$D'_{\vec{d}}(h) = \sum_{j=1}^{n} d_j D'_{\vec{e}^{(j)}}(h).$$

This idea is special enough that we give the terms $D'_{\vec{e}^{(j)}}(h)$ a special name: *partial derivatives*, and there is a slightly simpler way to calculate them.

- define partial derivatives
- define gradients
- relate both to directional derivatives

- with examples
- go again with convexity
- bring in hessian and eigenvalues as final condition